



國立臺灣科技大學工業管理系

碩士學位論文

學號：M11201305

神經演化強化學習於倉儲機器人交通控管策略

**Neuroevolutionary Reinforcement Learning for Traffic
Optimization in Robotic Mobile Fulfillment Systems**

研 究 生：蔡豐興

指導教授：周碩彥 博士

郭伯勳 博士

中華民國 114 年 6 月



碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form



M11201305

系所：工業管理系
Department/Graduate Institute Department of Industrial Management

姓名：蔡豐興
Name FONG HSING TSAI

論文題目：神經演化強化學習於倉儲機器人交通控管策略
(Thesis Title) Neuroevolutionary Reinforcement Learning for Traffic Optimization in Robotic Mobile Fulfillment Systems

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：

Advisor's Signature

周啟芳

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

郭伯勳

日期：

Date(yyyy/mm/dd)

2025 / 07 / 30



碩士學位考試委員審定書
Qualification Form by Master's Degree Examination Committee



M11201305

系所：工業管理系
Department/Graduate Institute Department of Industrial Management

姓名：蔡豐興
Name FONG HSING TSAI

論文題目：神經演化強化學習於倉儲機器人交通控管策略
(Thesis Title) Neuroevolutionary Reinforcement Learning for Traffic Optimization in Robotic Mobile Fulfillment Systems

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures

郭伯勳
周碩亨

郭中聖

指導教授簽章：

Advisor's Signature

周碩亨

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

郭伯勳

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature

工業管理系楊朝龍
系主任楊朝龍

日期：

Date(yyyy/mm/dd)

2025 / 07 / 30

論文摘要

本研究針對機器人移動履行系統 (RMFS) 中因交通壅塞所引發的「吞吐量-能源效率」權衡問題，提出一套基於神經演化強化學習 (NERL) 的自適應交通控管策略。為評估此策略在權衡空間中的定位，本研究於一高擬真度模擬平台，將 NERL 與代表靜態中心化(時間基礎)及去中心化(基於優先級)的兩種基線策略，在標準化的中等密度場景下進行了系統性的比較分析。

實驗結果量化了不同策略的性能取捨：去中心化策略雖達到最高的系統吞吐量，卻也伴隨著最高的能源消耗。相較之下，本研究提出的 NERL 策略在能源效率上展現顯著優勢，其單均能耗較去中心化策略降低了 19.4%，較時間基礎策略降低了 13.1%，同時維持了與時間基礎策略相當的訂單吞吐量。本研究的核心貢獻在於，證實了 NERL 控制器並非在所有指標上超越傳統方法，而是在「吞吐量-能耗」的權衡空間中，成功找到一個以能源效率為導向的新穎平衡點，為追求營運成本與可持續性的智慧倉儲系統，提供了一套有效的替代性優化方案。

關鍵字：自動化倉儲、機器人移動履行系統、交通控制、深度強化學習、神經演化、多目標優化

ABSTRACT

This study proposes an adaptive traffic control strategy based on **Neuroevolution Reinforcement Learning (NERL)** to address the critical trade-off between throughput and energy efficiency caused by traffic congestion in Robotic Mobile Fulfillment Systems (RMFS). To evaluate the positioning of this strategy within the trade-off space, this research conducted a systematic comparison of NERL against two baselines representing static centralized (Time-Based) and decentralized (Priority-Based) philosophies on a high-fidelity simulation platform under a standardized medium-density scenario.

The results quantify the distinct performance profiles: the decentralized strategy achieves the highest system throughput but at the cost of the highest energy consumption. In contrast, the proposed NERL strategy demonstrates significant advantages in energy efficiency, reducing the energy consumption per order by 19.4% compared to the decentralized strategy and 13.1% compared to the time-based strategy, while maintaining a comparable throughput to the time-based approach. The core contribution of this research is the empirical validation that the NERL controller successfully identifies a novel, energy-efficient operating point within the throughput-energy trade-off space. This provides an effective alternative optimization solution for intelligent warehouses prioritizing operational costs and sustainability.

Keywords: Automated Warehouse, Robotic Mobile Fulfillment System (RMFS), Traffic Control, Deep Reinforcement Learning (DRL), Neuroevolution, Multi-objective Optimization

ACKNOWLEDGMENTS

First and foremost, I would like to express my most sincere gratitude to my advisor, Professor Shuo-Yen Chou. Throughout my graduate studies, his invaluable guidance and insightful instruction have been instrumental in shaping this research. His rigorous academic standards and expansive vision have not only illuminated my path in research but have also set a profound example for me. I am also deeply grateful to my co-advisor, Professor Po-Hsun Kuo, for his consistent support and expert guidance, which greatly enriched the quality of this work.

My special thanks go to Professor Yu-Ling Hsu, a member of my oral defense committee, for her time, constructive feedback, and valuable suggestions that helped refine this thesis. I would also like to thank my classmates. Their generous help, collaborative spirit, and the insightful discussions we shared were a cherished and indispensable part of my academic journey. On a personal note, I wish to extend my deepest appreciation to my mother. Her unwavering support and encouragement have been my constant motivation to persevere through all the challenges. Finally, my heartfelt thanks to my girlfriend for her understanding, patience, and companionship, which have been my anchor throughout this demanding process. This thesis would not have been possible without the support of each and every one of them.

Fong Hsing Tsai

Taipei, 2025

TABLE OF CONTENTS

論文摘要	I
ABSTRACT	II
ACKNOWLEDGMENTS	III
CHAPTER 1 Introduction	1
1.1 Research Background and Motivation	1
1.2 Research Objectives	3
1.3 Research Scope and Assumptions	3
1.3.1 Research Scope	4
1.3.2 Core Assumptions for Focused Research	4
CHAPTER 2 Literature Review	6
2.1 Chapter Overview	6
2.2 Robotic Mobile Fulfillment Systems: Operational Efficiency and Traffic Bottlenecks	6
2.3 Review and Analysis of Intersection Traffic Control Methods	8
2.3.1 Heuristic and Rule-based Controllers	8
2.3.2 Deep Reinforcement Learning (DRL)-based Controllers	9
2.4 The Potential of Neuroevolution Reinforcement Learning (NERL)	10
2.5 Chapter Summary and Research Gap	12
CHAPTER 3 Methodology	15
3.1 Problem Definition	15
3.1.1 Chapter Structure	16
3.2 Experimental Environment and System Architecture	16
3.2.1 Warehouse Simulation Environment Design	17

3.2.2 Traffic Control System Architecture.....	22
3.2.3 Experimental Hardware and Software Configuration	25
3.2.4 Performance Evaluation Metrics Definition.....	27
3.2.5 Robot Physics Model and Energy Consumption.....	29
3.3 Baseline Control Strategies	33
3.3.1 Time-Based Controller	33
3.3.2 Priority-Based Decentralized Strategy	34
3.4 Neuroevolution Reinforcement Learning (NERL) Controller Design.....	35
3.4.1 NERL Evolutionary Process.....	36
3.4.2 State Space Design	38
3.4.3 Action Space Design.....	40
3.4.4 Global Reward Function Design	40
3.5 Experimental Design and Evaluation Process	42
3.5.1 Overview of the Experimental Process	42
3.5.2 Phase 1: Baseline Scenario Establishment	43
3.5.3 Phase 2: Baseline Controller Optimization	44
3.5.4 Phase 3: Final Performance Comparison Framework	45
3.6 NERL Training Process and Parameter Settings	46
3.6.1 Training Process	46
3.6.2 Neural Network Architecture.....	48
3.6.3 Hyperparameter Settings	48
3.6.4 Training Dynamics and Validity Verification	49
CHAPTER 4 Experimental Results and Analysis	53
4.1 Baseline Scenario Establishment: Optimal Robot Density Analysis.....	53
4.1.1 Experimental Setup	54

4.1.2 Results and Analysis: Seeking a Cost-Effectiveness Balance Point.....	54
4.1.3 Conclusion: Determining the Standard Experimental Density.....	55
4.2 Baseline Controller Optimization: Parameter Analysis of the Time-Based Controller.....	56
4.2.1 Experimental Setup	56
4.2.2 Results and Analysis.....	56
4.2.3 Conclusion: Determining the Optimized Baseline Controller.....	57
4.3 Final Performance Validation: Comparative Analysis of Three Control Philosophies.....	57
4.3.1 Comprehensive Performance Comparison	58
4.3.2 Throughput and Efficiency Analysis	60
4.3.3 Energy Consumption Analysis	62
4.3.4 Operational Efficiency and Decision Dynamics Analysis.....	63
4.3.5 Comprehensive Discussion: Analysis of the “Throughput-Energy” Trade- off Space	66
4.4 Chapter Summary	67
CHAPTER 5 Conclusion and Future Work.....	68
5.1 Research Summary	68
5.2 Research Contributions	69
5.3 Research Limitations	70
5.4 Future Work.....	71
References	74

LIST OF FIGURES

Figure 3.2.1 simulated warehouse	18
Figure 3.2.2 Robot Lifecycle.....	19
Figure 3.2.3 Critical Intersection red block.....	22
Figure 3.2.4 Intersection Manager Sequence Diagram	24
Figure 3.6.1 Evolution Trend of Best Fitness during NERL Training.....	50
Figure 4.3.1 Comparison of Order Completion Rate Distribution for Each Strategy	61
Figure 4.3.2 Comparison of Energy per Order Distribution for Each Strategy.....	62
Figure 4.3.3 Comparison of Robot Utilization Distribution for Each Strategy.....	64
Figure 4.3.4 Comparison of Signal Switch Count Distribution for Each Strategy	65

LIST OF TABLES

Table 2.5.1 Comparison with Related Research.....	14
Table 3.2.1 Robot Model Physical Parameters.....	31
Table 3.3.1 Priority levels.....	35
Table 3.4.1 State space	38
Table 3.6.1 Neural Network Architecture.....	48
Table 3.6.2 NERL hyperparameters	48
Table 3.6.3 Spearman’s Correlation Analysis between Generation Number and Fitness Indicators	50
Table 3.6.4 Spearman’s Correlation Analysis between Best Fitness and Core KPIs	51
Table 4.1.1 Performance and Cost-Effectiveness Analysis at Different Robot Densities	54
Table 4.2.1 Performance Comparison of Different Time Allocation Ratios(25 Robots)	56
Table 4.3.1 Final Model Validation Performance Comparison of Three Control Strategies	58
Table 4.3.2 Inter-group Statistical Comparison of Core Indicators.....	60
Table 4.3.3 Inter-group Statistical Comparison of Robot Utilization.....	63
Table 4.3.4 Inter-group Statistical Comparison of Signal Switch Count	65

CHAPTER 1

INTRODUCTION

1.1 Research Background and Motivation

The global retail industry is undergoing a structural transformation driven by e-commerce. Projections indicate that the annual value of global retail e-commerce will grow to \$6.42 trillion by 2025, accounting for over 20% of total retail sales [1]. To cope with the new normal of e-commerce orders—characterized by small batches, high frequency, and time sensitivity—traditional warehousing models are no longer adequate, prompting widespread adoption of **Robotic Mobile Fulfillment Systems (RMFS)**. In an RMFS, hundreds of Autonomous Mobile Robots (AMRs) work collaboratively within a grid-based warehouse, capable of increasing order picking efficiency severalfold.

However, high-density AMR operations have given rise to new operational bottlenecks. **Traffic congestion at intersections, waiting delays between robots, and frequent acceleration and deceleration behaviors** not only directly limit the system's overall throughput but also significantly increase energy consumption and carbon emissions. This issue has become critically important under the increasingly stringent global policies on sustainable development and carbon neutrality:

- **Significant Carbon Footprint:** Carbon emissions from transportation and warehousing activities account for approximately 24% of total global greenhouse gas emissions and are considered a priority area for carbon reduction by international logistics leaders like DHL [2].

- **Urgent Policy Pressure:** The European Union’s **Carbon Border Adjustment Mechanism (CBAM)**, initiated in 2023 and set to take full effect in 2026, will impose costs on high-carbon-intensity imported products. This will compel businesses throughout the supply chain to rigorously scrutinize and manage carbon emissions in their warehousing and transportation links [3].

Against this backdrop, academia has begun applying intelligent methods like Reinforcement Learning (RL) to optimize RMFS operations. Some studies have used deep reinforcement learning for order scheduling to minimize costs [4], while others have reduced total energy consumption by about 3–5% by adjusting traffic strategies and robot speeds [5]. However, these studies either focus on order-level scheduling or leave room for exploration in the trade-off between energy consumption and efficiency. More specifically, **there is still a lack of a control framework focused at the intersection level that aims to achieve the dual optimization goals of “energy-efficiency” at a micro-level by reducing unnecessary waiting and frequent “stop-and-go” behaviors.**

To fill this research gap, this study employs **Neuroevolution Reinforcement Learning (NERL)** to design an intersection controller. This method combines the global search capabilities of evolutionary algorithms with the real-time decision-making advantages of deep reinforcement learning. It aims to provide an “adaptive, energy-aware” right-of-way allocation mechanism for RMFS intersections, with the expectation of effectively reducing the energy consumption per task for AMRs while maintaining order processing efficiency. This is a direct technical approach to addressing the challenge of a high carbon footprint.

1.2 Research Objectives

The core objective of this study is to investigate and quantify the trade-off relationship between two conflicting goals—**system throughput** and **energy efficiency**—for different control strategies addressing traffic congestion in Robotic Mobile Fulfillment Systems (RMFS). The specific objectives are threefold:

1. **Design and Implement an Adaptive Controller:** Design and implement a traffic controller based on Neuroevolution Reinforcement Learning (NERL). The controller will utilize a global reward function aimed at balancing multiple macroscopic indicators, endowing it with the potential to autonomously learn complex trade-off strategies.
2. **Quantify the Performance of Different Control Philosophies:** Systematically compare the comprehensive performance of three strategies representing different control philosophies—the proposed **adaptive learning strategy (NERL)**, a **static centralized strategy (Time-Based)**, and a **decentralized strategy (No Control)**—under a standardized simulation environment.
3. **Analyze and Elucidate Performance Trade-off Characteristics:** Through in-depth analysis of experimental data, elucidate the relative positioning and characteristics of the three strategies within the “**throughput-energy**” **trade-off space**. This study aims to answer: Can NERL find a better comprehensive benefit balance point compared to traditional, single-objective-oriented strategies?

1.3 Research Scope and Assumptions

To ensure the depth and focus of this study, its research scope and corresponding assumptions are clearly defined as follows:

1.3.1 Research Scope

- **Problem Level:** This study focuses on **Intersection-level Traffic Control**. The core of the research is to coordinate the right-of-way for robots at intersections, without involving Global Path Planning for the robots.
- **Experimental Scenario:** All comparisons and analyses will be conducted in a standardized **Medium-density** warehouse scenario. This setting is designed to create a stable experimental environment that poses significant traffic challenges while avoiding system gridlock due to extreme congestion.
- **Evaluation Dimensions:** Performance evaluation will adopt a **Multi-objective Perspective**, focusing on analyzing the performance of each controller on the two core metrics of system throughput and energy efficiency, and their inherent trade-off.
- **Comparative Philosophies:** The core of this study is to compare three strategies representing different control **philosophies** (decentralized, static centralized, adaptive learning), aiming to deeply understand their intrinsic operational mechanisms and performance characteristics, rather than conducting a large-scale “audition” of algorithms.

1.3.2 Core Assumptions for Focused Research

To conduct an in-depth analysis of the intrinsic mechanisms of specific control strategies, this study is based on the following core assumptions to ensure consistency in comparison and depth of analysis:

- **Algorithm Breadth:** To deeply analyze the potential of NERL in trade-off learning, this study **will not conduct a horizontal performance comparison of**

various deep reinforcement learning algorithms (such as DQN, A3C, etc.).

- **Scenario Generalizability:** The conclusions of this study are based on a specific medium-density scenario. To ensure the depth of analysis, the performance of the controllers in higher or lower density scenarios, as well as their generalizability to different warehouse layouts, **are assumed to be beyond the scope of this study.**
- **Path Planning:** This study assumes that the paths of all robots are provided by an independent, default path planning system (such as the A* algorithm). The traffic controller's responsibility is solely to resolve local conflicts and **does not include** dynamically re-planning paths for robots.
- **Hyperparameter Optimization:** This study will use a fixed set of hyperparameter configurations for the NERL controller and **will not** conduct systematic Hyperparameter Tuning or Ablation Studies, to ensure all comparisons are made under a unified baseline.

CHAPTER 2

LITERATURE REVIEW

2.1 Chapter Overview

This chapter aims to systematically review the research related to traffic management in Robotic Mobile Fulfillment Systems (RMFS). It will unfold sequentially: first, by elaborating on the root causes of traffic congestion in RMFS and its profound impact on system performance; second, by evaluating three categories of control strategies relevant to this study—traditional heuristic methods, mainstream Deep Reinforcement Learning (DRL) methods, and emerging Neuroevolution methods—assessing their applications, achievements, and limitations in solving this problem. Through this structured analysis, this chapter aims to pinpoint the existing research gap and ultimately clarify the necessity and novelty of proposing a traffic control strategy based on Neuroevolution Reinforcement Learning (NERL) that integrates energy considerations.

2.2 Robotic Mobile Fulfillment Systems: Operational Efficiency and Traffic Bottlenecks

RMFS, as a “Goods-to-Person” paradigm for warehouse automation, fundamentally replaces traditional manual picking routes with a large number of Autonomous Mobile Robots (AMRs) that transport mobile shelves (Pods), thereby significantly enhancing order fulfillment efficiency [6]. Since Kiva Systems was acquired by Amazon and deployed on a massive scale [7], RMFS has become a cornerstone of modern e-commerce logistics. Studies show that its productivity can be two to four times that of traditional

warehouses. As the number of robots deployed by companies like Amazon exceeds 750,000, the operational density and complexity of these systems have reached unprecedented levels [8].

However, high-density deployment brings a new and severe bottleneck: **Traffic Congestion**. As defined in Chapter 3 of this study, when a large number of robots perform tasks within a limited aisle network, especially at intersections, conflicts and waiting periods easily arise from improper right-of-way coordination [9]. This triggers a series of cascading negative effects, not only causing delays in order fulfillment and reducing overall system throughput but also, as recent research indicates, the frequent stop-and-go behavior significantly increases overall energy consumption [10] and may accelerate robot battery aging, thereby increasing operational costs. Therefore, designing an **intersection management strategy** that is adaptive and considers both efficiency and energy sustainability has become a core challenge in enhancing the overall performance of RMFS.

In the complex decision-making framework of RMFS, traffic control and path planning are the most critical elements related to real-time system performance. In recent years, much research has been dedicated to solving this problem. For example, one study proposed [11] proposed a path planning method combining the A* algorithm with DQN, while another study used [12] used an attention-based DRL model to collaboratively handle multi-AGV scheduling and shelf reconfiguration. These studies highlight the immense potential of intelligent algorithms in RMFS operational optimization. Although this study focuses on intersection traffic control, its optimization goals—improving system fluidity and efficiency—are fundamentally aligned with these research efforts.

2.3 Review and Analysis of Intersection Traffic Control Methods

To address the problem of intersection traffic in RMFS, various control strategies have been proposed by academia and industry. Their development can be broadly categorized into heuristic methods and learning-based methods, reflecting a trend from static rules to dynamic, adaptive control.

2.3.1 Heuristic and Rule-based Controllers

Early solutions often relied on manually designed heuristic rules. Their advantages include simple implementation and low computational overhead, making them common baselines for evaluating the performance of more complex algorithms.

One category is **Static Time-based Control**, which borrows from traditional urban traffic signal systems by setting fixed cycles for switching right-of-way at intersections. This approach is conceptually similar to early rule-based urban traffic solutions [13] and corresponds to the **TimeBasedController** designed in Section 3.3.1 of this study. However, its fundamental flaw is its complete inability to adapt to real-time traffic flow changes. When traffic fluctuates dramatically, this static strategy is prone to wasting green light time on idle directions or causing unnecessary queue build-up in busy directions.

To overcome the shortcomings of static methods, another category, **Dynamic Reactive Control**, emerged. These methods react to the real-time state of the intersection (e.g., queue length). For instance, the work in assigned agent [14] assigned agents to intersections to manage traffic flow based on the priority of incoming robots. Although more flexible, the design of these heuristic rules is highly dependent on expert experience

and prior knowledge. Their predefined weights and logic struggle to capture the optimal coordination strategy in complex traffic patterns, often leading to sub-optimal performance in highly dynamic and non-linear traffic scenarios [15].

2.3.2 Deep Reinforcement Learning (DRL)-based Controllers

To overcome the limitations of manual rules, Deep Reinforcement Learning (DRL) has been widely applied to traffic control problems. DRL enables an agent (the controller) to autonomously learn complex decision-making policies through interaction with the environment to maximize long-term cumulative rewards [16].

This paradigm has achieved great success in the macroscopic field of **urban traffic signal control**. Numerous studies have demonstrated that methods based on Deep Q-Networks (DQN) and its variants significantly outperform traditional methods in reducing vehicle delays and improving network throughput [17]. For example, Cao et al. proposed [18] proposed G-DQN to improve convergence speed; Zhu et al. [19] compared the performance of PPO and DQN in single-intersection control; and Angela et al. [20] designed a reward function that incorporates multi-dimensional information such as pressure, queue, and speed to train DQN agents. These studies fully demonstrate the powerful capabilities of DRL in handling sequential decision-making problems.

In recent years, researchers have begun to transfer this paradigm to RMFS scenarios. They typically use local traffic information at intersections (e.g., queue length, waiting time) as the **State**, switching traffic phases as the **Action**, and maximizing throughput or minimizing waiting time as the **Reward** to train the controller [21] [22]. Although DQN-based methods show great potential, they also face some inherent challenges:

1. **Reward Design Sensitivity:** The performance of DQN is highly dependent on dense

and well-designed immediate reward signals. However, in complex trade-off problems like the one in this study, designing a perfect immediate reward is extremely difficult. A poorly designed or overly simplified reward function can lead the agent to learn unintended, sub-optimal behaviors (e.g., frequent signal flickering to gain switching rewards) as also noted in recent RL-based signal-control studies. [31] [33]

2. Risk of Local Optima: In complex, high-dimensional state spaces, gradient-based optimization methods can sometimes converge to a local optimum, failing to discover the globally optimal coordination strategy. [34]

3. Sample Efficiency Issues: DQN often requires a large number of environmental interactions to learn an effective policy, and the training process can be very time-consuming. [35]

These challenges, particularly the sensitivity of reward design and the risk of local optima, make it fundamentally difficult for DQN to handle the core “throughput-energy” multi-objective trade-off problem of this study. Designing a dense reward signal that can accurately reflect long-term macroscopic goals in real-time is extremely complex and can easily lead the policy into sub-optimal behaviors. It is to address these issues that this study turns to a learning paradigm more suited to handling sparse, delayed, and global rewards. These limitations have motivated scalable/federated variants of DQN and gradient-free EvoRL alternatives better suited to sparse or delayed rewards [31] [32] [33].

2.4 The Potential of Neuroevolution Reinforcement Learning (NERL)

To address the challenges faced by traditional DRL methods, this study introduces

another optimization paradigm—Neuroevolution Reinforcement Learning (NERL). Unlike DQN, which attempts to learn a value function, NERL searches directly in the **parameter space** of the policy network. It treats the weights of a neural network as an individual’s “genes” and uses evolutionary algorithms (such as elitism, crossover, and mutation) to optimize an entire population of agents, ultimately yielding a high-performance control policy [23].

NERL exhibits several unique advantages in solving complex control problems, making it particularly suitable for the objectives of this study:

1. **Robustness to Reward Functions:** Evolutionary algorithms are gradient-free, black-box optimization methods that do not require the reward function to be continuous or differentiable. This makes them naturally applicable to handling **sparse, delayed, or global performance-based reward signals**. For example, the “total number of completed orders” or “total energy consumption” over an entire simulation episode can be directly used as the fitness function. This aligns well with the **Global Reward** concept designed in Section 3.4.4 of this study, thus avoiding tedious Reward Engineering.
2. **Stronger Global Exploration:** The crossover and mutation operations in the evolutionary process facilitate broader exploration in the parameter space. By maintaining population diversity, the risk of getting stuck in local optima is reduced, making it more likely to discover innovative and efficient control strategies.
3. **Inherent Parallelism:** The evaluation of each individual (policy network) in the population is independent and can be deployed in parallel on multi-core CPUs or computing clusters. This allows for effective use of modern computing resources,

significantly reducing the wall-clock time for training [24].

Although neuroevolution has achieved remarkable success in fields like robotics control and game AI [25], the specific application of it to RMFS intersection traffic control, and a systematic comparison with traditional heuristic methods and mainstream DRL methods (like DQN) on a comprehensive platform that considers both time efficiency and energy consumption, remains a research gap.

2.5 Chapter Summary and Research Gap

Based on the analysis above, the development trajectory of existing RMFS traffic control research clearly points to several key unresolved issues. Control strategies are evolving from simple static rules to dynamic heuristics, and then to DRL-based adaptive methods. However, existing DRL methods (mainly DQN) still face challenges in reward design, avoiding local optima, and optimizing for long-term global objectives. Meanwhile, most traffic control research focuses on throughput and time delay, with a relative scarcity of studies that treat **energy consumption** as a core optimization target on par with efficiency.

More importantly, the potential of neuroevolution, as a method with stronger global exploration capabilities and greater robustness to sparse global rewards, has not been systematically explored and validated for the RMFS traffic control problem.

Therefore, the core research gap of this study can be summarized as: a lack of an intelligent control strategy for RMFS that can effectively balance and optimize traffic efficiency and energy consumption, while overcoming the limitations of traditional DRL in areas such as reward design and local optima.

To fill this gap, this paper proposes an intersection controller based on Neuroevolution Reinforcement Learning (NERL). We will conduct a comprehensive comparison of NERL with traditional heuristic methods and a standard DQN method on a high-fidelity simulation platform that incorporates a detailed energy consumption model. This research not only aims to propose a superior control strategy but also provides a solid empirical basis for evaluating the merits of different learning paradigms in solving complex multi-agent coordination problems.

To more intuitively compare this study with related literature, Table 2.1 summarizes the similarities and differences between selected representative studies and this research in terms of problem definition, decision criteria, and algorithmic appr

Table 2.5.1 Comparison with Related Research

Study	Performance Eval.	Energy Consumption	Dynamic Routing	Traffic Strategy	Multi-Agent	Heuristic	Reinforcement Learning
Li et al. (2020) [10]	✓	✓			✓		
Luo et al. (2023) [11]	✓		✓			✓	✓
Yuan et al. (2023) [26]	✓				✓		✓
Zhou et al. (2024) [27]	✓		✓	✓	✓		✓
Cao et al. (2024) [18]	✓			✓			✓
This Study (Ours)	✓	✓		✓	✓	✓	✓

CHAPTER 3

METHODOLOGY

3.1 Problem Definition

In modern automated warehouses utilizing Robotic Mobile Fulfillment Systems (RMFS), traffic congestion has become a core bottleneck constraining overall operational efficiency as order volumes and robot deployment densities increase. A large number of robots executing tasks within a limited aisle network, especially at intersections, can easily lead to conflicts and waiting due to improper right-of-way coordination. This triggers a series of cascading negative effects, including delays in order fulfillment, energy waste from robots waiting idly, and a decline in the system's overall throughput.

Therefore, the central problem of this research is: How to design an effective and self-adaptive traffic control strategy to dynamically regulate right-of-way at intersections, thereby minimizing robot waiting times and overall energy consumption, and ultimately enhancing the operational efficiency of the entire warehouse system?

To answer this question, it is essential to first construct an experimental platform that can accurately simulate the aforementioned challenges. The experimental environment and system architecture described in this chapter constitute a high-fidelity testbed tailored for this core problem. Its purpose is to provide a stable, controllable, and quantifiable foundation for the implementation, training, and comparison of different control algorithms in subsequent chapters.

3.1.1 Chapter Structure

This section aims to provide a comprehensive introduction to the experimental platform supporting this research. The content will cover four core parts: first, a detailed description of the constructed warehouse simulation environment, including its physical layout and core entities (Section 3.2.1); second, an explanation of the traffic control system architecture designed for algorithm evaluation (Section 3.2.2); third, a list of the specific hardware and software configurations used in the experiments (Section 3.2.3); and finally, the definition of the quantitative evaluation metrics used to measure the effectiveness of each control strategy (Section 3.2.4). Through this section, readers will gain a complete understanding of the foundational environment and evaluation criteria for the experiments.

3.2 Experimental Environment and System Architecture

To effectively validate and compare the Neuroevolution Reinforcement Learning (NERL) traffic control strategy proposed in this study with other baseline methods, it is necessary to construct a high-fidelity simulation platform that accurately reflects the challenges of real-world warehouse operations. This platform is not only a testing ground for algorithms but also a prerequisite for ensuring that all comparisons are conducted on a fair, controllable, and quantifiable basis.

This section will comprehensively describe the composition of this experimental platform from the inside out, from foundation to evaluation:

Simulation World Construction: First, we will detail the two foundations of the

simulation world—the **physical environment design of the warehouse (Section 3.2.1)** and the **physics and energy consumption model (Section 3.2.5)** that endows the robots with realistic energy characteristics. These two elements together define the “physical laws” of the experiment.

Control System Design: Next, we introduce the **traffic control system architecture (Section 3.2.2)** designed for flexible algorithm integration, which serves as the “central nervous system” of the experiment.

Evaluation Framework and Environment: Finally, to ensure the objectivity and reproducibility of the research, we will clearly define the **performance evaluation metrics (Section 3.2.4)** used to measure effectiveness and list the **hardware and software configurations (Section 3.2.3)** used in the experiments.

Through this section, readers will gain a complete understanding of the experimental environment, control logic, and evaluation standards of this study.

3.2.1 Warehouse Simulation Environment Design

To effectively evaluate traffic control strategies, this study first constructed a high-fidelity warehouse simulation environment. This environment not only defines the physical layout but also includes various dynamic entities and their interaction rules, which together form a complex Robotic Mobile Fulfillment System (RMFS). This section will detail its design.

1. Physical Environment and Layout

The simulated warehouse is built on a two-dimensional discretized grid, with each grid cell having a specific function. The overall layout adopts a functional zoning design to

ensure an orderly operational flow.

- **Central Storage Area:** Located in the center of the warehouse, it consists of a dense arrangement of **Pod Locations**. The aisles in this area are designed as strict **One-way Aisles**, with the flow directions of horizontal and vertical aisles alternating. This design greatly simplifies traffic management complexity at the physical level, aiming to reduce potential conflicts from robots moving in opposite directions.
- **Workstation Area:** Distributed on both sides of the warehouse. One side houses **Picking Stations**, which are the exit points for order fulfillment, and the other side has **Replenishment Stations**, the entry points for goods into the system.
- **Charging Stations:** Scattered within the storage area, converted from some pod locations, for robots to charge autonomously

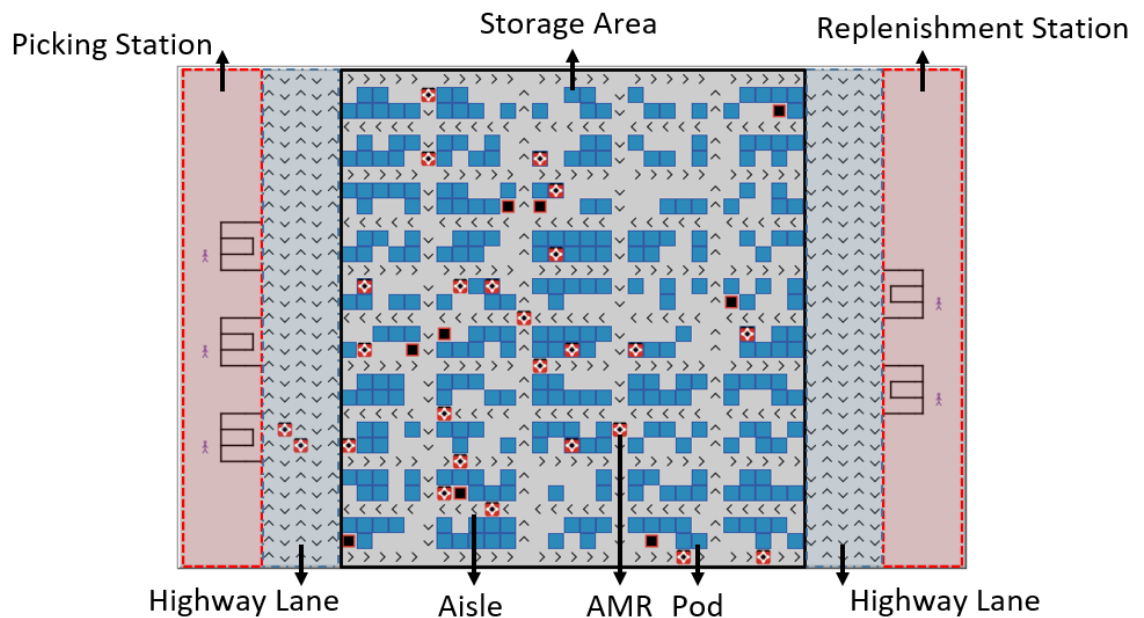


Figure 3.2.1 simulated warehouse

2. Core Entities and Lifecycles

The dynamic behavior of the system is driven by the interactions among several core entities.

- Robot:** As the most central active unit in the system, the robot has a complex state machine to manage its workflow, including states like `idle`, `taking_pod`, `delivering_pod`, `station_processing`, and `returning_pod`. This study establishes a detailed physics and energy model for the robot. Its energy consumption calculation considers not only the load but also startup costs and regenerative braking (energy recovery during braking), providing a solid basis for energy efficiency evaluation. Additionally, robots have a priority-based autonomous obstacle avoidance logic, enabling them to resolve local conflicts to some extent.

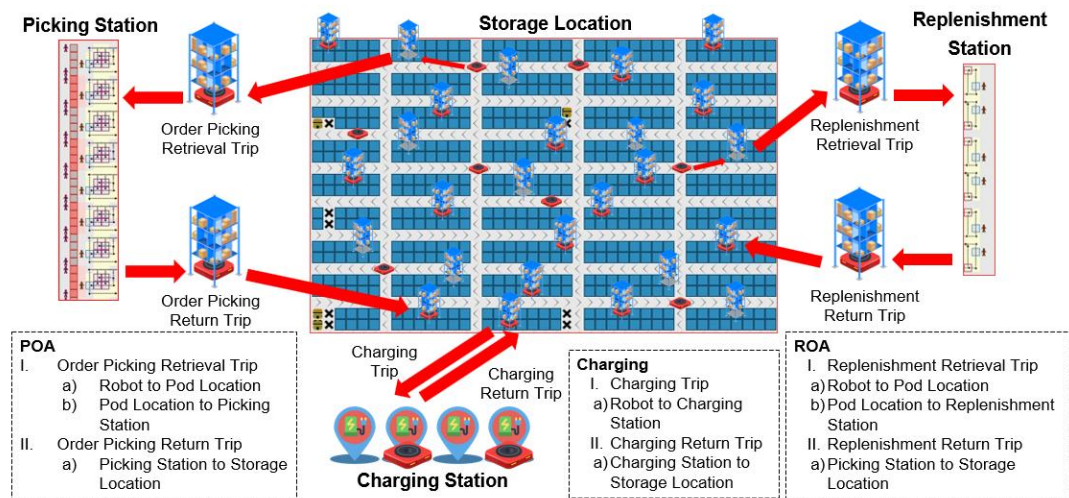


Figure 3.2.2 Robot Lifecycle

- Pod:** The mobile carrier for storing goods (SKUs). Each pod can hold multiple types of SKUs and records the current quantity and replenishment threshold for each SKU. When the stock level falls below the threshold, the system automatically triggers a corresponding replenishment task.

- **Station:** The node for human-robot collaboration. When a robot delivers a pod to a station, the system simulates the delay of a worker picking or replenishing goods. To handle high traffic, stations are also designed with a dynamic path adjustment mechanism, activating a longer backup path to alleviate congestion when there are too many robots at the station.

3. Order and Task Flow

The driving force of the simulation comes from orders. An **Order** represents a customer's request and contains multiple SKUs to be picked. The system breaks down an order into one or more **Jobs**. The core of a job is “transporting a specified pod to a specified workstation,” which is the smallest work unit that can be directly assigned to a robot.

The entire process is as follows:

1. The system receives an order.
2. The SKUs required for the order are located on specific pods.
3. The system generates one or more jobs and places them in the job queue.
4. An idle robot takes a job from the queue and begins its work cycle of picking up, delivering, and returning the pod.
5. When all SKUs for an order have been successfully delivered to the workstation, the order is marked as complete.

4. Intersection Design and Classification

In addition to the macroscopic layout, this study provides a clear classification and definition of the microscopic traffic nodes within the warehouse—the intersections. This is crucial for the subsequent controller design.

- **Standard Intersection:** This is the basic unit that constitutes the main traffic network of the warehouse, formed by the convergence of two mutually perpendicular one-way aisles. All intersections not otherwise specified fall into this category.
- **Critical Intersection:** Based on their strategic importance in the warehouse layout, some intersections are marked as “critical.” These are traffic nodes on the entry or exit paths directly connected to **workstations (picking or replenishment stations)**. They are the mandatory routes for entering and exiting the workstations and constitute the primary **bottlenecks** of the entire warehouse system. The efficiency of managing these intersections will directly affect workstation throughput and robot queue lengths, and may trigger **spillback** traffic into the storage area. Therefore, in the design of the reinforcement learning reward function (see Section 3.4.4), these intersections will be given higher weights to guide the agent to prioritize their effective management

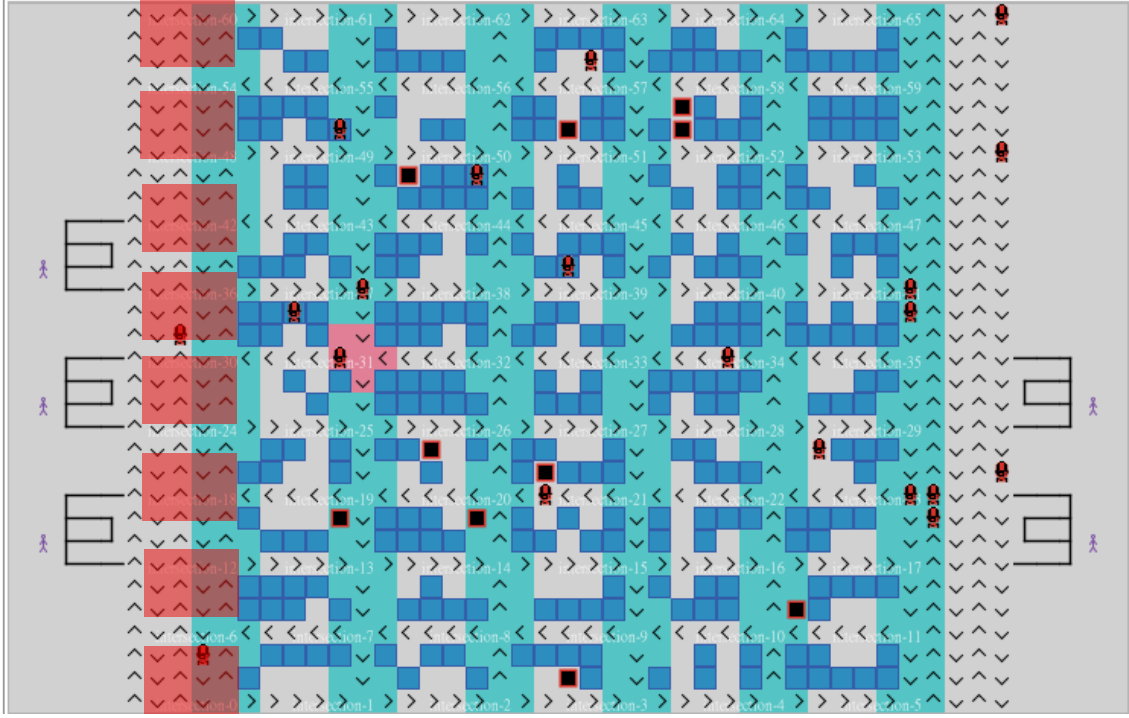


Figure 3.2.3 Critical Intersection red block

3.2.2 Traffic Control System Architecture

To enable flexible integration and fair comparison of different traffic control algorithms, this study designed a modular software architecture based on the **Strategy Pattern** and **Factory Pattern**. The core of this architecture is to separate the “decision-making algorithm” from the “system execution framework,” ensuring that whether it’s a simple rule-based logic or a complex deep reinforcement learning model, it can operate and be evaluated on the same foundation. The system is mainly composed of the following three components:

1. TrafficController (Abstract Base Class for Traffic Controllers)

This Abstract Base Class (ABC) defines a unified interface that all traffic control strategies must adhere to. Its core method is `get_direction(intersection, tick, warehouse)`, which receives the current detailed state of the intersection (local information) and the

state of the entire warehouse system (global information), and returns a traffic decision for that intersection (e.g., "Horizontal" or "Vertical"). By forcing all controllers to implement this interface, the system ensures consistency in how different algorithms are called. Additionally, the base class integrates standardized statistical data collection functions to record various performance metrics.

2. TrafficControllerFactory (Controller Factory)

This class uses the factory design pattern and is responsible for dynamically creating instances of TrafficController subclasses based on external configurations (such as the controller type specified in the experiment configuration file). When the simulation core needs a controller, it only needs to provide a string identifier like "dqn", "nerl", or "time_based", and the factory will return a corresponding, initialized controller object. This design completely decouples the “creation logic” from the “usage logic” of the controllers, greatly enhancing the flexibility and scalability of the experimental process, allowing for switching between different control strategies without modifying any core simulation code.

3. IntersectionManager (Intersection Manager)

The IntersectionManager is the central coordinator and execution engine of the entire traffic control system. Its operational flow forms a complete closed-loop control system:

1. **Holds Controller Instance:** During the simulation initialization phase, the IntersectionManager obtains the required controller instance for the current experiment through the controller factory.
2. **Drives Decision Loop:** In each time unit (tick) of the simulation, the manager

iterates through all intersections in the warehouse.

3. **Gets Decision:** For each intersection, it calls the `get_direction()` method of the `TrafficController` instance to get the traffic command for that intersection.

4. **Executes Decision:** Based on the command returned by the controller, the `IntersectionManager` updates the internal state of the intersection, such as changing the allowed direction of passage.

5. **Triggers Model Training:** Specifically, for reinforcement learning-based controllers (DQN/NERL), after the decision and execution steps are completed, the `IntersectionManager` will then call its `train()` method, providing the model with the just-occurred state transition (State-Action-Reward-NextState), enabling it to learn and optimize from experience.

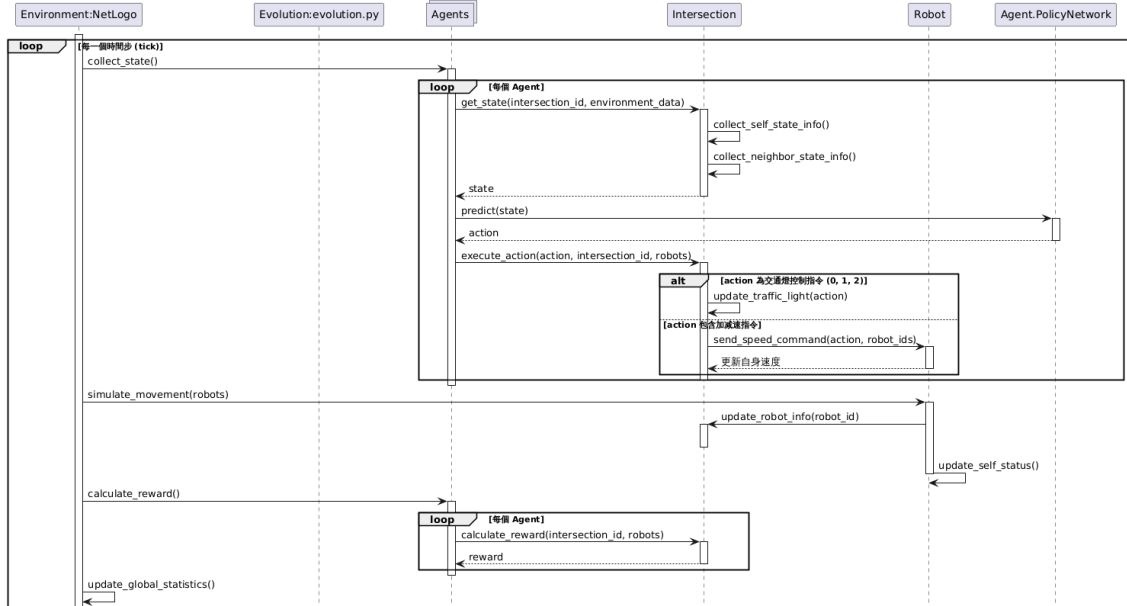


Figure 3.2.4 Intersection Manager Sequence Diagram

3.2.3 Experimental Hardware and Software Configuration

To ensure the reproducibility and validity of the results of this study, all simulation, training, and evaluation experiments were conducted on a well-defined hardware platform and in a standardized software environment. This section will detail the relevant configurations.

Hardware Configuration

The computational tasks of this study were mainly divided into two parts: model training on a high-performance cloud platform, and development, debugging, and result analysis on a local machine.

Training Environment (Runpod Secure Cloud)

All computationally intensive model training tasks were performed on the Runpod cloud platform to leverage its powerful computing resources and accelerate the learning process.

- **CPU:** 42 vCPUs
- **GPU:** 1 x NVIDIA GeForce RTX 4090
- **Memory (RAM):** 83 GB

Development and Analysis Environment (Laptop)

Program development, initial testing, parameter tuning, and final data analysis and visualization were completed on a local computer.

- **CPU:** AMD Ryzen 9 6900HX
- **GPU:** NVIDIA GeForce RTX 3080 Ti

- **Memory (RAM):** 16.0 GB

Software Configuration

The choice of software environment was aimed at balancing development efficiency, computational performance, and broad community support.

Training Environment: Ubuntu 22.04 (in a runpod/pytorch:2.1.0-py3.10-cuda11.8.0-devel Docker container)

Development Environment: Microsoft Windows 11

Programming Language: Python 3.10

Core Computing Libraries:

- PyTorch (version 2.1.0): As the core deep learning framework, used for building and training DQN and NERL neural network models.
- numpy (version 1.24.4): Provided the foundation for all numerical computations, widely used in state representation, reward calculation, etc.
- pandas (version 2.0.3): Mainly used for processing, storing, and analyzing experimental data.

Simulation and Analysis Tools:

- networkx (version 3.2.1): Used for constructing and analyzing the warehouse's network graph structure.
- scikit-learn (version 1.5.2): In this study, mainly used for data preprocessing steps like normalization.

Visualization Libraries:

- matplotlib (version 3.7.2): Used for generating static 2D charts, such as line graphs and bar charts.
- seaborn (version 0.13.2): Based on matplotlib, provided more aesthetically pleasing and statistically-oriented visualizations, such as heatmaps.

System Tools:

- psutil (version 5.9.8): Used for monitoring system resource usage.

3.2.4 Performance Evaluation Metrics Definition

To objectively and quantitatively evaluate the pros and cons of different traffic control strategies, this study established a comprehensive set of Key Performance Indicators (KPIs). For clarity, we first define the following mathematical symbols: - R : The set of all robots in the warehouse. - $O_{\text{completed}}$: The set of all completed orders during the simulation period. - P : The set of all robot passing events at intersections. - T_{sim} : The total simulation duration, in *ticks*.

1. Efficiency Metrics

Total Energy Consumption This metric measures the overall energy efficiency of the system and is one of the core optimization targets of this study. It is calculated as the sum of the energy consumed by all robot activities during the simulation (see Section 3.2.5 for the detailed calculation model).

$$E_{\text{total}} = \sum_{r \in R} E_r (3 - 1)$$

where E_r represents the total energy consumption of a single robot r throughout the simulation, in energy units (EU).

2. Throughput Metrics

Completed Orders Count This metric directly measures the total output of the system within a fixed time, reflecting the overall operational efficiency.

$$N_{\text{orders}} = |O_{\text{completed}}| \quad (3-2)$$

Average Order Processing Time This metric measures the system's response speed in handling a single order. It is defined as the average time taken for all completed orders from the start of processing to completion.

$$T_{\text{avgorder}} = \frac{1}{|O_{\text{completed}}|} \sum_{o \in O_{\text{completed}}} (t_{\text{complete}}(o) - t_{\text{start}}(o)) \quad (3-3)$$

where $t_{\text{complete}}(o)$ and $t_{\text{start}}(o)$ are the completion and start times of order o , respectively, both in *ticks*.

Average Intersection Waiting Time This metric directly reflects the coordination efficiency of the traffic control strategy. It calculates the average waiting time for each robot passing event at an intersection.

$$W_{\text{avg}} = \frac{1}{|P|} \sum_{p \in P} t_{\text{wait}}(p) \quad (3-4)$$

where $|P|$ is the total number of times robots pass through intersections, and $t_{\text{wait}}(p)$ is the waiting time for a single passing event p , in *ticks*.

3. Stability Metrics

Total Stop-and-Go This metric reflects the smoothness of the traffic flow. Frequent stops and starts not only consume extra energy but also indicate unstable traffic flow.

$$S_{\text{total}} = \sum_{r \in R} N_{\text{s-g}}(r) \quad (3 - 5)$$

where $N_{\text{s-g}}(r)$ is the number of stop-and-go events for robot r .

3.2.5 Robot Physics Model and Energy Consumption

To ensure the authenticity and credibility of the simulation, this study established a robot model based on physical principles. This section will detail the unit system used in the simulation, the core parameters of the robot, and the energy consumption calculation model.

1. Unit System Definition

To unify the physical calculations in the simulation, this study adopts the following consistent unit system:

- **Time:** The basic time unit in the simulation is the *tick*. According to the system settings, each *tick* corresponds to 0.15 seconds in the real world.
- **Distance:** One unit of length in the simulation corresponds to 1 meter in the real world.
- **Mass:** The unit of mass in the simulation is kilograms (kg).
- **Velocity:** The unit is meters per second (m/s).
- **Acceleration:** The unit is meters per second squared (m/s²).
- **Energy:** Energy is calculated based on standard physical formulas. To maintain consistency within the simulation, we define its unit as “Energy Unit (EU),” which is proportional to the Joule (J).

2. Robot Physical Parameters

The following table details the physical parameters applied to all robot entities. These parameters are defined in `world/entities/robot.py` and remain constant throughout the simulation.

Table 3.2.1 Robot Model Physical Parameters

Parameter	Symbol	Value	Unit	Physical Meaning
Robot Mass	m_{robot}	1	kg	Mass of the robot itself
Load Mass	m_{load}	0	kg	Additional load when the robot is carrying goods
Gravity	g	10	m/s ²	Approximate value of Earth's gravitational acceleration
Friction Coeff.	μ	0.3	-	Rolling friction between the simulated ground and wheels
Inertia Coeff.	I	0.4	-	Inertial resistance experienced by the robot during acceleration
Startup Cost	C_{startup}	0.5	EU	Additional energy cost to overcome static friction when starting from rest
Regen. Braking Eff.	η_{regen}	0.3	-	Proportion of kinetic energy that can be recovered during braking (30%)
Maximum Speed	v_{max}	1.5	m/s	Maximum travel speed of the robot without restrictions

3. Energy Consumption Calculation Model

The energy consumption model in this study not only considers basic movement energy but also integrates startup costs and a regenerative braking mechanism to more realistically reflect the energy characteristics of electric robots. Total energy consumption is calculated by accumulating the energy used in different motion states of the robot.

A. Constant Velocity Motion Energy When a robot moves at a constant velocity v ,

its energy consumption is mainly from overcoming friction.

$$E_{\text{friction}} = (m_{\text{robot}} + m_{\text{load}}) \cdot g \cdot \mu \cdot v \cdot \Delta t \quad (3 - 6)$$

where Δt is one time step (0.15 seconds).

B. Accelerated Motion Energy When a robot accelerates with an acceleration a , it needs to overcome inertia in addition to friction.

$$E_{\text{accel}} = (m_{\text{robot}} + m_{\text{load}}) \cdot (g \cdot \mu + a \cdot l) \cdot \bar{v} \cdot \Delta t \quad (3 - 7)$$

where $\bar{v}$ is the average velocity within that time step.

C. Startup Cost When a robot starts moving ($v > 0$) from a standstill ($v_{\text{prev}} = 0$), a one-time, fixed startup cost is incurred.

$$E_{\text{startup}} = C_{\text{startup}} \quad (3 - 8)$$

D. Regenerative Braking (Energy Recovery) When a robot brakes to a complete stop ($v = 0$) from moving ($v_{\text{prev}} > 0$), the system recovers a portion of the energy based on its change in kinetic energy.

$$E_{\text{regen}} = \frac{1}{2} (m_{\text{robot}} + m_{\text{load}}) \cdot v_{\text{prev}}^2 \cdot \eta_{\text{regen}} \quad (3 - 9)$$

E. Total Energy Change Combining all the above items, the total energy change ΔE_{total} of a robot in a single time step can be expressed as:

$$\Delta E_{\text{total}} = E_{\text{movement}} + E_{\text{startup}} - E_{\text{regen}} \quad (3 - 10)$$

where E_{movement} corresponds to E_{friction} or E_{accel} depending on whether the robot is in constant velocity or accelerated motion. The total energy consumption of the system is the cumulative sum of ΔE_{total} for all robots over all time steps.

3.3 Baseline Control Strategies

To objectively evaluate the performance of the Neuroevolution Reinforcement Learning (NERL) traffic control method proposed in this study, it must be compared against a set of representative Baseline Strategies. These baseline strategies provide a performance reference point, allowing us to quantify the actual improvements brought by the complex algorithm. This study selects two distinct control philosophies as baselines: a **Time-Based Strategy** representing traditional static centralized management, and a **No-Control Strategy** representing fully decentralized decision-making.

3.3.1 Time-Based Controller

The TimeBasedController is the most basic static traffic control strategy. Its core idea is derived from traditional urban traffic signal systems, completely ignoring real-time traffic flow or any dynamic changes at the intersection. It relies solely on a pre-set, fixed time cycle to switch the right-of-way between horizontal and vertical directions. The advantage of this method is its extreme simplicity and predictability, but its disadvantage is equally obvious—it cannot adapt to fluctuations in traffic demand, easily causing unnecessary waiting during busy times or wasting passage time during light traffic.

Design Principles and Decision Logic

The operation of this controller is determined entirely by two core parameters: the green light time for the horizontal direction (T_{H_green}) and the green light time for the vertical direction (T_{V_green}). The sum of these two constitutes a complete signal cycle length (T_{cycle}).

$$T_{cycle} = T_{H_green} + T_{V_green} \quad (3 - 11)$$

At any given time tick in the simulation, the controller determines the current position in the signal cycle (t_{pos}) using a modulo operation:

$$t_{pos} = \text{tick} \bmod T_{cycle} \quad (3 - 12)$$

Based on the value of t_{pos} , the controller decides the direction of passage. If t_{pos} is less than the horizontal green time T_{H_green} , it grants the right-of-way to the horizontal direction; otherwise, it grants it to the vertical direction.

$$\text{Direction} = \begin{cases} \text{Horizontal,} & \text{if } t_{pos} < T_{H_green} \\ \text{Vertical,} & \text{if } t_{pos} \geq T_{H_green} \end{cases} \quad (3 - 13)$$

The parameters of this controller will be systematically optimized in the experimental design of Section 3.5.3 to find its best performance configuration.

3.3.2 Priority-Based Decentralized Strategy

To establish a performance baseline, this study designed a priority-based decentralized strategy. This strategy can also be viewed as a “No-Central-Controller” scenario, intended to simulate an environment without any central traffic coordination mechanism. In this setting, the right-of-way at intersections is determined entirely by the robots themselves based on a simple, predefined set of priority rules. This strategy represents an important performance floor, used to measure the net benefit of any centralized control measure.

Design Principles and Decision Logic

When multiple robots have a potential conflict at an intersection, the system assigns a priority to each robot based on its current task state. The priority levels are defined in

the table below, with the core principle being that tasks directly related to order fulfillment (such as delivering a pod to a workstation) have higher priority.

Table 3.3.1 Priority levels

Robot State (current_state)	Task Description	Priority
delivering_pod	Delivering pod to picking station	Highest
returning_pod	Returning pod to storage area	High
taking_pod	Going to storage area to pick up a pod	Medium
idle	Idle or waiting	Lowest

At an intersection, the robot with the higher priority gets the right-of-way. This model, based entirely on individual greedy decisions, may perform well at low densities but is a source of high energy consumption and potential deadlock risk.

3.4 Neuroevolution Reinforcement Learning (NERL)

Controller Design

To apply the Neuroevolution Reinforcement Learning framework to the traffic control problem of this study, we first establish the mapping of its core elements, which are detailed in the following subsections:

- **Agent:** The **traffic controller** at each intersection.
- **Environment:** The **high-fidelity warehouse simulation system** as described in **Section 3.2.1**.

- **State:** The feature vector observed by the agent at the time of decision, its detailed composition is in **Section 3.4.2**.
- **Action:** The discrete commands the agent can execute, its detailed composition is in **Section 3.4.3**.
- **Reward:** a **global fitness score** calculated only at the end of an evaluation episode, its design philosophy and formula are detailed in **Section 3.4.4**.

To cope with the high dynamics and complexity of traffic within RMFS and to learn to trade-off between multiple conflicting objectives like “throughput” and “energy consumption,” this study employs **Neuroevolution Reinforcement Learning (NERL)** to develop an intelligent traffic controller.

Unlike traditional gradient-based reinforcement learning methods (like DQN), NERL optimizes directly in the **parameter space** of the policy. In this framework, the weights and biases θ of a neural network controller (i.e., a policy π_θ) are treated as an individual’s **Genotype**. The algorithm searches for the policy network parameters θ^* that maximize the system’s long-term macroscopic benefits by performing “evaluation-selection-reproduction” evolutionary operations on a **Population** of many individuals. This method is more robust to sparse, delayed global reward signals and is naturally suited for large-scale parallel training [25] [29].

This section will detail the core evolutionary process of the NERL controller, the state space, the action space, and the crucial design of the global reward function.

3.4.1 NERL Evolutionary Process

The core process of NERL revolves around an “evaluation $\rightarrow$ selection $\rightarrow$ reproduction” evolutionary cycle, which iterates to improve the overall performance of

the population. Assume at generation g there is a population $P_g = \{\theta_1, \theta_2, \dots, \theta_N\}$, containing N individuals.

1. **Evaluation:** Each individual θ_i in the population is deployed in an independent simulation environment instance to run a full evaluation episode. During the episode, the policy π_{θ_i} makes all decisions independently. After the episode ends, a **Fitness Score** $F(\theta_i)$ is calculated for the individual based on the system's macroscopic performance metrics (detailed in Section 3.4.4).
2. **Selection:** Based on the calculated fitness scores of all individuals, the algorithm selects high-performing individuals from the current population P_g as **Parents** to provide genes for the next generation. This study uses **Tournament Selection**.
3. **Reproduction:** A new offspring population P_{g+1} is generated by applying genetic operations to the parents. The main operations include:
 - **Elitism:** To prevent losing the best solutions found during evolution, the top k elite individuals with the highest fitness in each generation are directly and completely copied to the next generation's population.
 - **Crossover:** Simulating biological reproduction, two individuals are selected from the parents, and their parameter vectors are mixed to create new offspring.
 - **Mutation:** A small random perturbation is applied to the parameters of the offspring individuals to maintain population diversity and avoid premature convergence.

By iteratively executing the above cycle, the average fitness of the population will gradually increase, eventually converging to a high-performance policy network that can efficiently solve complex traffic control problems.

3.4.2 State Space Design

The definition of the state space is a cornerstone of success in reinforcement learning. It must provide the agent with sufficient and meaningful information to make effective decisions. In the RMFS traffic control problem of this study, the state s_t of any intersection at decision time t is defined as a feature vector that comprehensively reflects its local traffic conditions while also considering the potential impact of neighboring and downstream intersections.

Specifically, s_t is a **17-dimensional continuous vector**, which, after being processed by Adaptive Normalization, is divided into the following three parts:

Table 3.4.1 State space

State Type	Dimension	Feature Name	Description
Local State	1	Current Direction	Indicates the current green light direction (horizontal/vertical)
	1	Time Since Change	The duration the current signal phase has been active
	2	Queue Lengths (H/V)	The number of queuing robots in each of the two directions
	2	Priority Ratios (H/V)	The proportion of robots in each queue performing high-priority tasks (e.g., delivering a pod)
	2	Average Wait Times (H/V)	The average waiting time of robots in each queue

State Type	Dimension	Feature Name	Description
Neighbor State	1	Neighbor Count	The total number of adjacent intersections (1-4)
	1	Total Neighbor Queue	The total number of queuing robots at all neighboring intersections
	1	Total Neighbor Priority Queue	The total number of robots performing high-priority tasks at all neighboring intersections
	1	Neighbor Priority Ratio	The overall proportion of high-priority task robots at neighboring intersections
	1	Neighbor Average Wait Time	The average waiting time at all neighboring intersections
	1	Neighbor Phase Distribution	The proportion of neighboring intersections that are in horizontal and vertical green light phases, respectively
	1	Neighbor Load Balance	A metric for the disparity in queue lengths among neighboring intersections

Note: Local state has 8 dimensions, neighbor state has 8 dimensions, totaling 16 dimensions, plus 1 dimension for global state (explained below), for a total of 17 dimensions.

3. Global State - 1 Dimension: A macroscopic indicator is introduced to help the agent align local decisions with overall system goals. * **Picking Station Queue:** The total queue length at the entrances of all picking stations, a key indicator of system-level bottlenecks.

State Normalization

Before feeding the state vector s_t into the NERL model, this study employs an **Adaptive Normalization** technique. This technique dynamically tracks the running mean and standard deviation of each state feature during training and uses them to standardize the feature values, thereby ensuring the stability and efficiency of model training.

3.4.3 Action Space Design

This study designs a concise discrete action space $\mathcal{A}$ for the DRL agent. At each decision point, the controller can choose an action $a_t \in \mathcal{A}$ from a set of **6 discrete actions**. These actions cover basic traffic signal phase control and dynamic speed limits.

- **Basic Phase Control (3 actions):** Keep Current Phase, Switch to Vertical Green, Switch to Horizontal Green.
- **Dynamic Speed Control (3 actions):** Remove Speed Limit (resume normal speed), Set Speed Limit to 50%, Set Speed Limit to 30%.

All DRL controllers operate at a fixed decision interval of $T_{\text{decision}} = 10$ ticks.

3.4.4 Global Reward Function Design

The reward function (often called the fitness function in evolutionary algorithms) is the core element that guides the agent's behavior. To guide the NERL controller to learn complex trade-off strategies that are beneficial to the system's long-term, macroscopic goals, this study designed a comprehensive **Global Reward Function**. In this model, the agent receives no immediate feedback throughout the evaluation episode. A single fitness score is calculated only at the end of the episode, based on the final overall performance of the system.

The design of this function incorporates established principles from the field of multi-objective optimization [30] and includes customized penalty terms (such as a long-tail delay penalty) tailored to the unique challenges of the RMFS scenario in this study, to more comprehensively capture the system's overall performance. Its core formula is defined as follows:

$$\text{Fitness} = (N_{\text{orders}} \cdot w_{\text{orders}}) - (E_{\text{total}} \cdot w_{\text{energy}}) - \sum_{i \in P} (\lambda_i \cdot \text{Penalty}_i) \quad (3 - 14)$$

where N_{orders} is the total number of completed orders, E_{total} is the total energy consumption, and w_{orders} and w_{energy} are their corresponding weights. $\sum(\lambda_i \cdot \text{Penalty}_i)$ is a series of weighted penalty terms used to penalize undesirable system behaviors.

Specifically, the penalty term P includes the following key parts:

- **Maximum Wait Time Penalty (P_{wait}):** Penalizes the maximum robot waiting time that occurred throughout the simulation to encourage fairness and prevent individual robots from starving.
- **Average Wait Time Penalty ($P_{\text{avg wait}}$):** Penalizes the average time of all waiting events to improve overall traffic fluidity.
- **Stop-and-Go Penalty (P_{sg}):** Penalizes the total number of stop-and-go events in the system to encourage smooth traffic flow and indirectly reduce energy consumption.
- **Underutilization Penalty (P_{util}):** Penalizes situations where the average robot utilization falls below a certain reference value to encourage the system to maintain high operational efficiency.
- **Long-Tail Delay Penalty (P_{tail}):** Penalizes high-percentile waiting times (e.g., P95/P99) to specifically address and improve the worst experiences in the system.

All penalty terms are **normalized** based on a preset threshold before calculation, constraining them to the $[0, 1]$ interval to ensure that penalties of different scales can be stably combined. This multi-objective, penalty-normalization-based fitness function design aims to guide the evolutionary process to find a comprehensively optimal solution that is not only high-throughput but also energy-efficient and highly stable.

It is important to emphasize that the effectiveness of this reward function is not proven through theoretical derivation but will be empirically validated by the experimental results in Chapter 4. If the NERL controller trained based on this function can demonstrate the expected intelligent trade-off behavior between multiple conflicting objectives in the final performance evaluation, then this design can be considered successful.

3.5 Experimental Design and Evaluation Process

To rigorously answer the research questions posed in the introduction, this study designed a systematic three-phase experimental process. This process is intended to ensure that all control strategies are compared on a fair and controllable basis, and to clearly reveal the performance differences between different control philosophies.

3.5.1 Overview of the Experimental Process

The overall validation process follows these three core steps:

1. **Phase 1: Baseline Scenario Establishment** First, a series of preliminary simulations are conducted to determine a **standard robot density** that can generate representative traffic pressure. This density will serve as a uniform configuration for all subsequent formal experiments to ensure consistency in

comparison.

2. **Phase 2: Baseline Controller Optimization** Next, a systematic **parameter sensitivity analysis** is performed on the traditional **Time-Based Controller**. This step aims to find an optimal set of traffic light cycle parameters for it, to ensure that the subsequent comparison with NERL is made against a well-optimized baseline.
3. **Phase 3: Final Performance Comparison** Finally, under the standardized experimental scenario, a comprehensive, multi-dimensional performance evaluation and comparison is conducted on the three strategies representing different control philosophies—NERL, the **optimized Time-Based Controller**, and the **decentralized (no-control) strategy**.

3.5.2 Phase 1: Baseline Scenario Establishment

To establish a standardized testbed that can effectively evaluate controller performance, the core objective of this phase is to determine a “**medium-pressure**” **robot density**. If the density is too low, it will not generate enough traffic conflicts to distinguish between the merits of the controllers; if it is too high, the system may quickly become saturated under any control strategy, also preventing effective comparison.

- Experimental Method:
 - The **decentralized (no-control) strategy** is used as the testing baseline.
 - The number of robots in the simulation is systematically varied, for example, starting from 20 and increasing in increments of 5 up to 40.
 - For each density setting, 20 complete, long-duration simulations of 100,000 ticks are run repeatedly.

- Evaluation Metrics:
 - The main observation metrics are total system throughput (number of completed orders) and average energy consumption per order.
- Decision Criteria:
 - The number of robots at which significant performance bottlenecks begin to appear (e.g., throughput growth slows while energy consumption continues to rise) is selected as the standard configuration for all subsequent experiments.

3.5.3 Phase 2: Baseline Controller Optimization

To ensure a fair comparison with the baseline, this phase aims to find the optimal parameter configuration for the **Time-Based Controller**.

- Experimental Method:
 - Experiments are conducted at the standard robot density determined in Phase 1.
 - A systematic parameter sweep is performed on the two core parameters of the time-based controller—horizontal green time and vertical green time. For example, while keeping the total cycle length constant (e.g., 100 ticks), different time allocation ratios (e.g., 90/10, 80/20, ..., 20/80) are tested.
 - Each parameter combination is run multiple times in simulation.
- Evaluation Metrics:
 - The main observation metrics are order completion rate and total energy consumption.
- Decision Criteria:

- The parameter configuration that performs best overall on the above two metrics is selected as the version of the time-based controller for the final comparison phase.

3.5.4 Phase 3: Final Performance Comparison Framework

This phase is the core validation part of this study. A comprehensive horizontal comparison of the three final control strategies is conducted under the determined standard scenario.

- Comparison Subjects:
 1. **NERL Controller**: The final model after sufficient training, using the architecture and global reward function described in Section 3.4.
 2. **Time-Based Controller (Optimized)**: Using the best parameter configuration found in Section 3.5.3.
 3. **Decentralized Strategy (No Control)**: As a baseline reference for performance.
- Evaluation Process:
 - All experimental groups are run multiple times (e.g., 20 times) under the **same random seed** to eliminate random interference and obtain statistically significant results.
 - Each evaluation is a complete, long-duration simulation (e.g., 100,000 ticks).
- Core Comparison Dimensions:
 - System Throughput: With order completion rate and total completed orders as primary indicators.

- Energy Efficiency: With total energy consumption and Energy per Order as primary indicators.
- Operational Stability: With metrics like total stop-and-go count and average intersection waiting time as auxiliary analysis.
- Analysis Method:
 - The average KPI data for all experimental groups are compiled into a comprehensive performance comparison table.
 - A “throughput-energy” scatter plot is drawn to visually represent the positioning of different strategies in the trade-off space.
 - Appropriate statistical tests are used to verify the significance of differences in key indicators.

3.6 NERL Training Process and Parameter Settings

To ensure the reproducibility of the research results, this section details the training process and key hyperparameters used for the NERL controller. The parameter configuration listed here was determined based on a series of preliminary experiments on convergence and stability. This study does not claim this configuration to be the global optimum but presents it as a standardized setup that can stably train high-performance models and reliably reproduce the core findings of this paper.

3.6.1 Training Process

The training of NERL is a generation-iterative, off-policy learning process. A single complete training experiment follows this procedure:

1. Initialization:

- a. An instance of NEController is created according to the hyperparameters described in Section 3.6.2.
 - b. The controller randomly initializes a population of 30 network individuals.
2. Evolutionary Loop:
 - a. An evolution is started for 20 generations.
 - b. In each generation g :
 - i. **Parallel Evaluation:** For the 30 individuals in the population, 30 independent simulation environments are launched.
 - ii. Each individual runs a full evaluation episode of 25,000 ticks in its dedicated environment. To ensure consistency in learning objectives, all evaluations are conducted under a fixed order demand; however, a different random seed is used for each evaluation to change initial conditions (like Pod layout), thereby generating diverse, dynamic training scenarios for the controller through **real-time interaction** with the simulator.
 - iii. After the episode, the fitness score of the individual is calculated according to the global reward function defined in Section 3.4.4.
 - iv. **Evolutionary Operation:** Once the fitness scores of all individuals are calculated, the controller performs a full evolutionary operation (selection, crossover, mutation) to generate a new offspring population.
3. Model Saving:
 - a. At the end of each generation, the algorithm saves the individual with the highest fitness in that generation as the best model of that generation.
 - b. After the evolution of all generations is complete, the model with the

highest fitness score across all generations is selected as the final model for the final performance comparison in Section 3.5.4.

3.6.2 Neural Network Architecture

The neural network used by the NERL controller in this study is a standard Feed-Forward Neural Network. This architecture strikes a balance between the model’s expressive power and computational efficiency. Its layer definitions are as follows:

Table 3.6.1 Neural Network Architecture

Layer	Type	Input Dim	Output Dim	Activation
1	Input	17	128	ReLU
2	Hidden 1	128	64	ReLU
3	Output	64	6	- (Linear)

The input layer dimension (17) corresponds to the state space defined in Section 3.4.2, and the output layer dimension (6) corresponds to the action space defined in Section 3.4.3.

3.6.3 Hyperparameter Settings

The following table details the key hyperparameters and their default values used in the training process of the NERL controller.

Table 3.6.2 NERL hyperparameters

Hyperparameter	Code Variable	Default Value	Description
Population Size	population_size	30	A common trade-off between computational resources and population diversity.
Elite Ratio	elite_ratio	0.2	The top 20% of individuals with the highest fitness in each generation are directly preserved, ensuring the best solution is not lost.
Tournament Size	tournament_size	4	The number of individuals randomly compared in each tournament selection.
Crossover Rate	crossover_rate	0.8	The probability of two parent individuals performing genetic crossover.
Mutation Rate	mutation_rate	0.2	The base probability of an individual's genes (network weights) mutating.
Mutation Strength	mutation_strength	0.15	The standard deviation of the Gaussian mutation, controlling the magnitude of the mutation.

3.6.4 Training Dynamics and Validity Verification

To ensure the internal consistency and validity of the methodology of this study, it is necessary to analyze the training process of NERL itself. This section aims to answer two key questions: First, does the evolutionary process show an **overall upward trend** towards optimization? Second, does the global fitness function designed in Section 3.4.4 successfully guide the agent to learn the expected trade-off behavior?

1. Trend Analysis of the Evolutionary Process

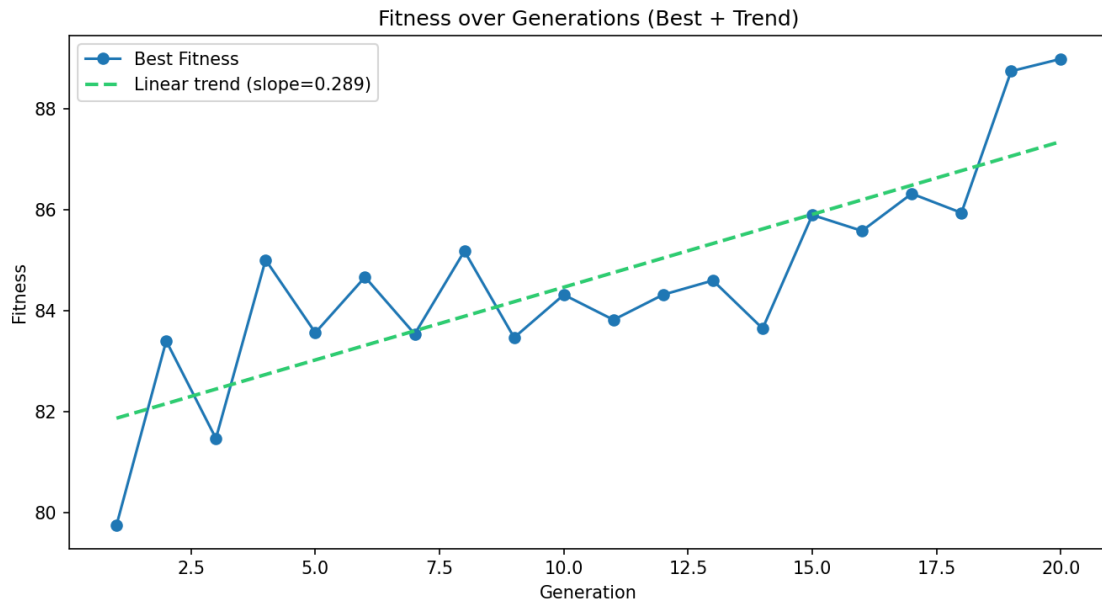


Figure 3.6.1 Evolution Trend of Best Fitness during NERL Training

Figure 3.6.1 shows the change in the fitness score of the best individual (Elite) in each generation over 20 generations of training. As the figure shows, although there are normal fluctuations in the evolutionary exploration, the overall improvement trend is very clear. Linear regression analysis shows that the best fitness score has a positive growth trend with the number of generations (slope = 0.289, $R^2 = 0.657$). For further rigorous statistical validation, we conducted a Spearman's rank correlation analysis between the number of generations and various fitness indicators, with the results shown in Table 3.6.3.

Table 3.6.3 Spearman's Correlation Analysis between Generation Number and Fitness Indicators

Metric	Correlation Coeff. (ρ)	p-value	q-value (FDR)
Best Fitness	0.808	< 0.001	< 0.001

Metric	Correlation Coeff. (ρ)	p-value	q-value (FDR)
Mean Fitness	0.478	0.033	0.049
Median Fitness	0.232	0.326	0.326

The analysis results provide strong data support for the effectiveness of the algorithm. There is a statistically highly significant strong positive correlation between the generation number and the **best fitness score** ($\rho = 0.808$, $q < 0.001$), indicating that the performance improvement trend of the algorithm is not a random coincidence. At the same time, the **population mean fitness** also shows a statistically significant positive trend ($\rho = 0.478$, $q = 0.049$), which proves that the performance improvement is a collective progress at the population level, not just relying on a few lucky individuals. Overall, this training process successfully guided the population to explore towards a better policy space, providing a reliable final model for the performance comparison in Chapter 4.

2. Validity Verification of the Fitness Function

To verify whether the fitness function designed in this study successfully guided the agent to learn the expected multi-objective trade-off behavior, we conducted a Spearman's rank correlation analysis between the fitness score of the best individual of each generation and its corresponding Key Performance Indicators (KPIs). The results are summarized in Table 3.6.4.

Table 3.6.4 Spearman's Correlation Analysis between Best Fitness and Core KPIs

Key Performance Indicator (KPI)	Correlation Coeff. (ρ)	p-value	q-value (FDR)
Completed Orders	0.614	0.004	0.036
Stop-and-Go Count	0.501	0.025	0.110
Avg. Order Time	-0.531	0.016	0.096
Robot Utilization	-0.338	0.146	0.365
Energy per Order	-0.347	0.134	0.365

Note: Statistical methods are the same as in Table 3.6.3.

The results of the Spearman's rank correlation analysis (Table 3.6.4) provide profound insights into the design direction of the fitness function. First, after rigorous multiple comparison correction, the best fitness shows a statistically significant positive correlation with the **number of completed orders** ($\rho = 0.614$, $q = 0.036$). This indicates that the evolutionary process has indeed effectively optimized towards the core goal of improving system throughput.

It is noteworthy that we observe a negative correlation between the best fitness score and **average order processing time** ($\rho = -0.531$, $q = 0.096$), and a positive correlation with the **total stop-and-go count** ($\rho = 0.501$, $q = 0.110$). Although these two indicators did not reach statistical significance with the current sample size, their trends reveal an important insight: **the evolutionary process is not simply sacrificing the efficiency of single orders to gain overall throughput. On the contrary, the data trend suggests that a better controller seems to have found a path of synergistic optimization. While improving the number of completed orders**($\rho = 0.614$, $q = 0.036$), they also tend to

reduce the average order processing time. This indicates that the fitness function has successfully guided the model to learn a deeper level of operational efficiency, not just crudely increasing task parallelism. Although the increasing trend of stop-and-go counts may reflect more complex traffic management at high throughput, overall, the model shows the potential to make progress on multiple key objectives simultaneously.

At the same time, the negative correlation between the fitness score and **energy per order** ($\rho = -0.347$, $q = 0.365$) also did not reach statistical significance, but its direction is consistent with the energy-saving goals of this study. Overall, the data indicate that our designed reward function successfully guided the NERL controller to explore in the “throughput-energy” trade-off space towards the desired “high-throughput, low-energy” direction. Therefore, the final model produced by this training process is a reasonable and effective representative for the performance comparison in Chapter 4.

CHAPTER 4

EXPERIMENTAL RESULTS AND ANALYSIS

4.1 Baseline Scenario Establishment: Optimal Robot Density Analysis

Following the experimental procedure outlined in Chapter 3, this section aims to complete the first phase: establishing a standardized baseline scenario for the effective evaluation of different control strategies. The core objective is not to pursue the system’s maximum throughput but to identify an “optimal operating point” that balances representative traffic pressure with the best “cost-effectiveness,” which will serve as a

uniform configuration for all subsequent comparative experiments.

4.1.1 Experimental Setup

To achieve this goal, this study used the priority-based decentralized strategy as the testing baseline. Under this strategy, the system's performance purely reflects the relationship between the physical environment and robot density. The specific experimental parameters were set as follows:

- **Controller:** Priority-based decentralized strategy.
- **Test Variable:** Number of robots, starting from 20 and increasing in increments of 5 up to 40.
- **Simulation Duration:** Each density setting was run for 100,000 ticks.
- **Repetitions:** To ensure statistical significance, each density setting was repeated in 20 independent experiments.

4.1.2 Results and Analysis: Seeking a Cost-Effectiveness

Balance Point

This section will analyze the marginal benefits and costs at different robot densities, focusing on the two core dimensions of system throughput and energy consumption.

Table 4.1.1 Performance and Cost-Effectiveness Analysis at Different Robot Densities

Number of Robots	Average Completed Orders	Order Growth Rate	Average Energy per Order	Energy Consumption Growth Rate
20	415.7	-	173.8	-
25	450.7	+8.4%	216.7	+24.7%
30	452.7	+0.4%	246.1	+13.6%
35	448.6	-0.9%	264.8	+7.6%

As shown in Table 4.1.1, increasing the number of robots from 20 to 25 brought the most significant benefit increase: a 25% increase in robots resulted in an 8.4% growth in orders. However, when the density was increased from 25 to 30 robots, the order growth rate plummeted to just 0.4%, while the average energy consumption per order still rose sharply by 13.6%. This marks the “cost-effectiveness inflection point” of the system: the additional robot resources were mostly consumed by internal traffic friction and did not translate into effective output. When the density was further increased to 35 robots, the system even experienced a lose-lose situation of decreased throughput and continued energy consumption increase. Similar density–throughput trade-offs are observed in recent RL-based traffic-signal studies [31][32].

4.1.3 Conclusion: Determining the Standard Experimental Density

Based on the cost-effectiveness analysis above, this study will select **25 robots** as the standard density configuration for all subsequent comparative experiments. This configuration represents the optimal efficiency point of the system before saturation,

providing sufficient traffic pressure while maximizing the energy efficiency of each operation, making it an ideal baseline scenario for evaluating the “cost reduction and efficiency improvement” capabilities of different control strategies.

4.2 Baseline Controller Optimization: Parameter Analysis of the Time-Based Controller

After determining the standard experimental density, the experimental process moves to the second phase. This section aims to perform a systematic parameter sensitivity analysis and optimization for the traditional Time-Based controller.

4.2.1 Experimental Setup

- **Controller:** Time-Based Controller.
- **Experimental Scenario:** The standard density of **25 robots** determined in Section 4.1.
- **Test Variable:** A parameter sweep of the allocation ratio for horizontal/vertical green light times.
- **Simulation Duration and Repetitions:** Each parameter combination was run 20 times, each for 100,000 ticks.

4.2.2 Results and Analysis

To find the optimal parameters, this study tested various time allocation ratios. The table below summarizes the core performance of each parameter combination at a density of 25 robots.

Table 4.2.1 Performance Comparison of Different Time Allocation Ratios(25 Robots)

Parameter (H:V)	Order Rate	Completion	Robot Utilization	Energy per Order
60:40	92.7%		95.1%	194.9
75:25	92.1%		95.3%	201.2
65:35	92.0%		94.1%	195.3
80:20	91.4%		94.0%	197.9
50:50	91.3%		91.2%	198.4

As shown in Table 4.2.1, the 60:40 time allocation ratio achieved the best or near-best performance on multiple key indicators, including order completion rate, robot utilization, and energy per order, demonstrating the best overall benefits.

4.2.3 Conclusion: Determining the Optimized Baseline Controller

Based on the analysis above, this study will select the **60:40** time allocation ratio as the **optimized Time-Based Controller** for the final performance comparison in the third phase.

4.3 Final Performance Validation: Comparative Analysis of Three Control Philosophies

This section is the core of the entire experimental study, aiming to complete the third phase: a comprehensive performance comparison and in-depth analysis of the three control philosophies under the standardized scenario of 25 robots. To ensure the fairness

of the evaluation and the generalization ability of the models, the experimental design in this phase follows these core principles:

To ensure all strategies are compared under the **same Demand Pressure**, this study used a **fixed total number of orders and order arrival schedule** in all training and validation runs. However, this does not mean the training and validation environments were identical. On the contrary, to test the generalization ability of the models, each validation run used a **different Random Seed**.

This leads to changes in the crucial initial conditions for the controller, such as the **initial layout of Pods** and the **starting positions of the robots**. Therefore, although the macroscopic order demand remained consistent, the specific task sequences, robot paths, and the resulting traffic flows at intersections were **unique and unseen** by the controller in each validation scenario. This design fundamentally prevents the possibility of the model merely “memorizing” or “overfitting” to specific solutions, ensuring that what is evaluated in this section is the controller’s **Generalization Ability** in the face of unknown traffic conditions.

4.3.1 Comprehensive Performance Comparison

For a macroscopic comparison, the following table summarizes the final validation data of the three control strategies on five core KPIs. All values are the average results of 20 repeated experiments under the standard density.

Table 4.3.1 Final Model Validation Performance Comparison of Three Control Strategies (25 Robots)

Control Strategy	Order Completion Rate	Completed Orders	Energy per Order	Total Energy	Signal Switches
Decentralized	97.4%	451	215.4	97,142	N/A
Time-Based (60:40)	93.7%	433	199.6	86,369	19,800
NERL (Global Reward)	93.4%	429	173.5	74,530	29,835

From the macroscopic data in the summary table, a clear trade-off relationship between “throughput” and “energy efficiency” can be observed for the different control strategies.

The priority-based decentralized strategy demonstrated the highest system throughput with an order completion rate of **97.4%** and **451** completed orders. However, this high efficiency came at the cost of significant energy consumption, with its energy per order being the highest among the three at **215.4**.

In contrast, the NERL strategy proposed in this study excelled in energy efficiency. Its energy per order was only **173.5**, which is a significant reduction of **19.4%** and **13.1%** compared to the decentralized and time-based strategies, respectively. This advantage makes it the most energy-efficient option. However, this energy optimization was achieved by sacrificing some system throughput, as its number of completed orders (429) was noticeably lower than that of the decentralized strategy.

The performance of the parameter-optimized time-based controller fell between the other two, offering a relatively balanced but not optimal solution. Overall, the core contribution of the NERL strategy is that it reveals the possibility of significantly

optimizing energy efficiency through intelligent decision-making, although this comes with a certain degree of capacity trade-off in the current experimental setup. Subsequent statistical tests will analyze these differences in more detail.

4.3.2 Throughput and Efficiency Analysis

To more rigorously evaluate the performance differences between the strategies, the following table summarizes the descriptive statistics (mean M, standard deviation SD) of the core indicators and the p-values from inter-group T-tests.

Table 4.3.2 Inter-group Statistical Comparison of Core Indicators

Indicator		NERL (M ± SD)	Time-Based (M ± SD)	Decentralized (M ± SD)	p-value (NERL vs. Time-Based)	p-value (NERL vs. Decentralized)
Order Completion Rate (%)		93.4 ± 3.6	93.7 ± 1.7	97.4 ± 1.8	0.760	<0.001
Energy per Order		173.5 ± 31.1	199.6 ± 10.0	215.4 ± 10.2	0.002	<0.001

Note: $p < 0.05$ indicates a statistically significant difference.

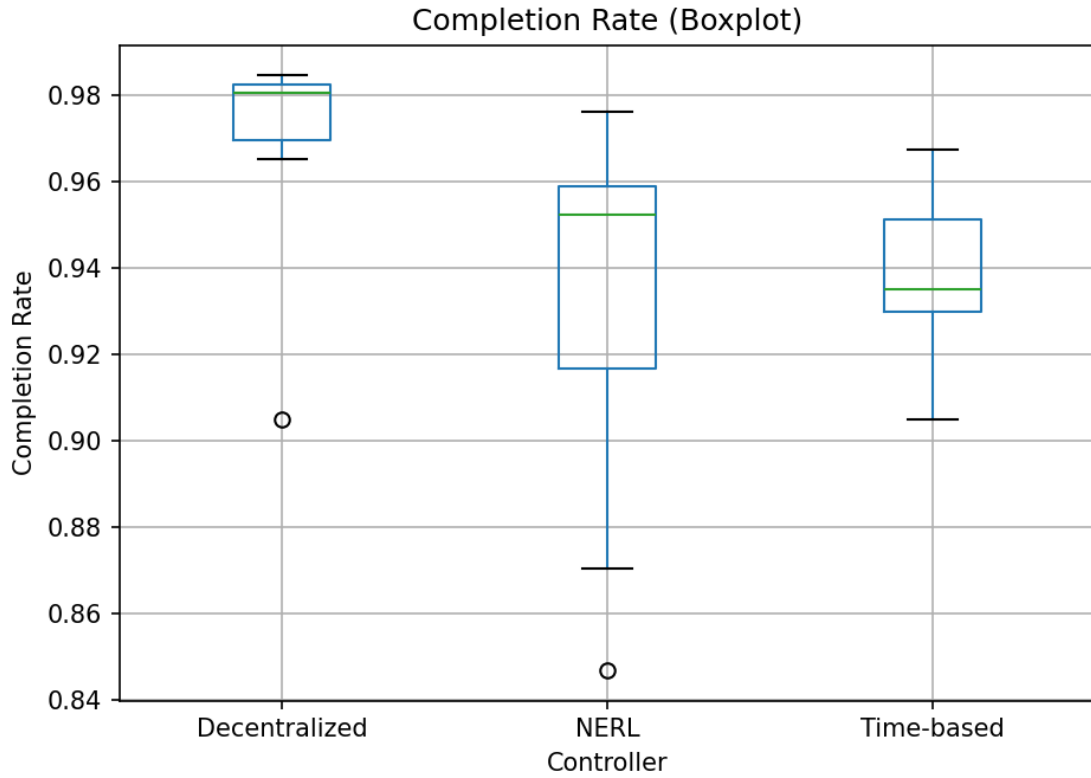


Figure 4.3.1 Comparison of Order Completion Rate Distribution for Each Strategy

This subsection will delve into the differences in core output metrics among the three strategies. As shown in **Table 4.3.2**, the decentralized strategy achieved an average order completion rate of **97.4%**, and the T-test results show that its lead is **highly statistically significant** ($p < 0.001$ compared to NERL). On the other hand, the completion rates of the NERL and time-based strategies were very similar (93.4% and 93.7%, respectively), with no statistically significant difference between them ($p = 0.760$). The box plot in **Figure 4.3.1** also visually confirms this, with the data distribution of the decentralized strategy being overall higher than the other two, while the data distribution intervals of NERL and the time-based strategy significantly overlap. The pattern matches network-level improvements reported for RL-based controllers [31] [32].

This result indicates that the decentralized strategy remains the optimal choice for

maximizing system throughput. However, NERL and the traditional time-based strategy can achieve a relatively stable and acceptable level of output. This finding makes the subsequent comparison of energy consumption particularly crucial, as it will reveal the potential of the NERL strategy in “cost reduction.”

4.3.3 Energy Consumption Analysis

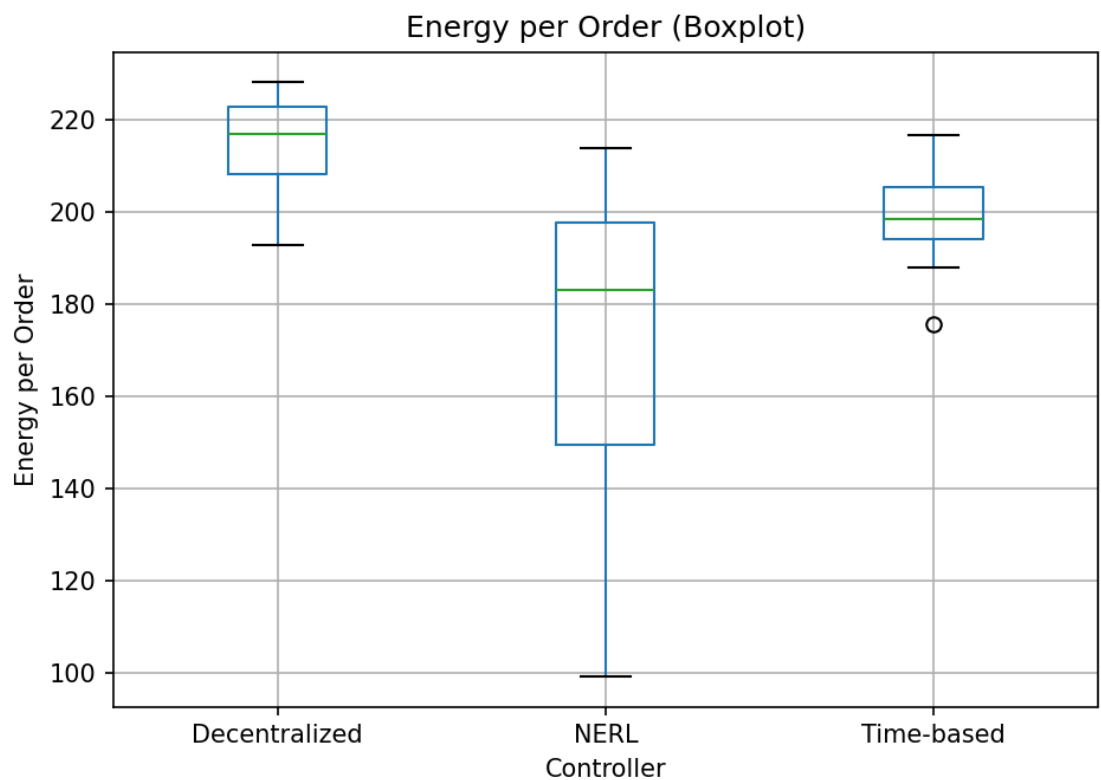


Figure 4.3.2 Comparison of Energy per Order Distribution for Each Strategy

In terms of energy efficiency, NERL’s advantage is clear and strongly supported by statistical data. According to **Table 4.3.2**, the energy per order of the NERL strategy is not only significantly lower than the time-based strategy ($p = 0.002$) but also significantly better than the decentralized strategy, which has the highest throughput ($p < 0.001$). The box plot in **Figure 4.3.2** also clearly shows that the data distribution of NERL is

significantly lower overall than the other two. Energy–throughput trade-offs are also emphasized in RMFS energy-aware studies [11].

Specifically, compared to the **decentralized strategy** that pursues maximum throughput, NERL significantly reduced the energy per order by **19.4%** (from 215.4 to 173.5). Compared to the parameter-optimized **time-based controller**, it also saved **13.1%** of energy (from 199.6 to 173.5). This result strongly demonstrates that the adaptive coordination strategy learned by NERL can, while sacrificing some capacity, successfully create a smoother traffic flow through dynamic adjustment of traffic signals, thereby effectively reducing energy waste caused by conflicts, waiting, and frequent stop-and-go events.

4.3.4 Operational Efficiency and Decision Dynamics Analysis

In addition to the core throughput and energy consumption metrics, this section will further explore the deeper differences in operational efficiency and decision dynamics among the different control strategies by analyzing robot utilization and the number of signal switches.

First, **Table 4.3.3** summarizes the inter-group statistical data for robot utilization. Comparable trends are seen in multi-agent TSC benchmarks [32][35].

Table 4.3.3 Inter-group Statistical Comparison of Robot Utilization

Indicator	NERL (M $\pm$ SD)	Time-Based (M $\pm$ SD)	Decentralized (M $\pm$ SD)	p-value (NERL vs. Time-Based)	p-value (NERL vs. Decentralized)
Robot Utilization	96.9 $\pm$ 2.2	96.6 $\pm$ 1.7	93.3 $\pm$ 3.2	0.691	<0.001

Indicator	NERL ($M \pm SD$)	Time-Based ($M \pm SD$)	Decentralized ($M \pm SD$)	p-value (NERL vs. Time-Based)	p-value (NERL vs. Decentralized)
(%)					

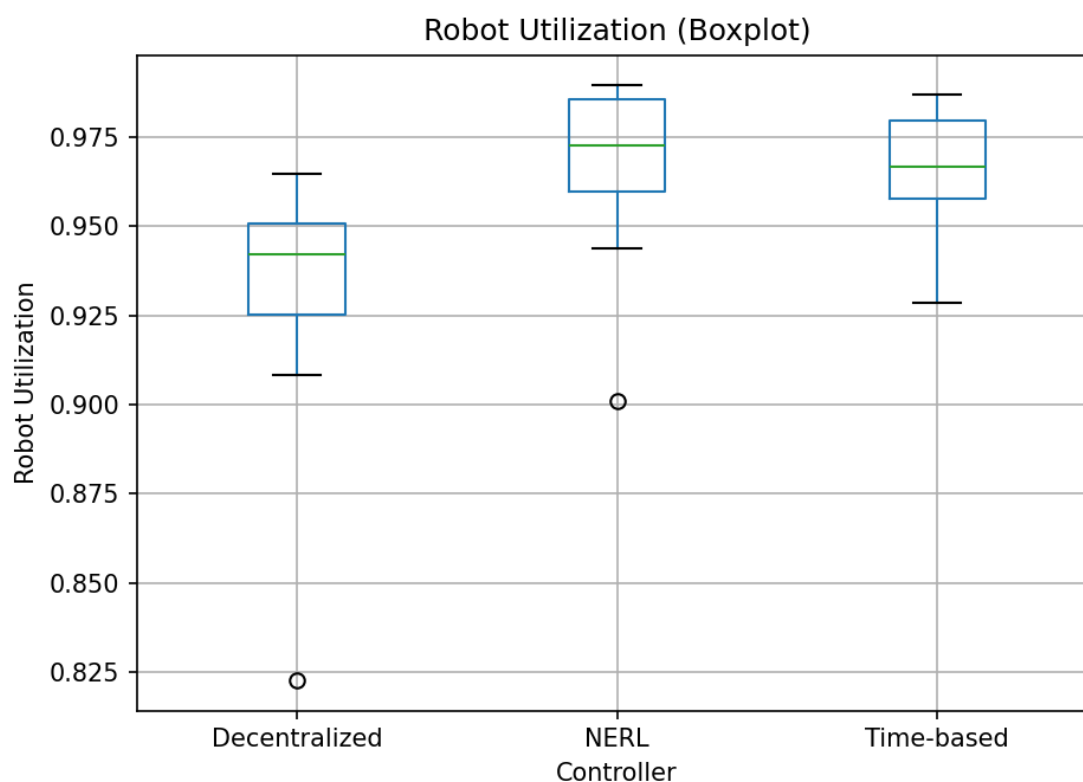


Figure 4.3.3 Comparison of Robot Utilization Distribution for Each Strategy

The data shows that both NERL and the time-based controlled strategies exhibited similar and high levels of utilization, with no statistically significant difference between them. However, both were significantly higher than the decentralized strategy. This indicates that although the decentralized strategy has the highest order processing capacity, the overall work saturation of its robot fleet is actually lower, possibly due to poor path planning or local congestion leading to longer idle times for some robots. In contrast, the controllers were able to more effectively allocate tasks and right-of-way, keeping the

entire fleet in a busier and more efficient operational state.

Second, **Table 4.3.4** reveals the “decision activity” of the different controllers.

Table 4.3.4 Inter-group Statistical Comparison of Signal Switch Count

Indicator	NERL (M ± SD)	Time-Based (M ± SD)	p-value (NERL vs. Time-Based)
Signal Switch Count	29,835 ± 753	19,800 ± 0	<0.001

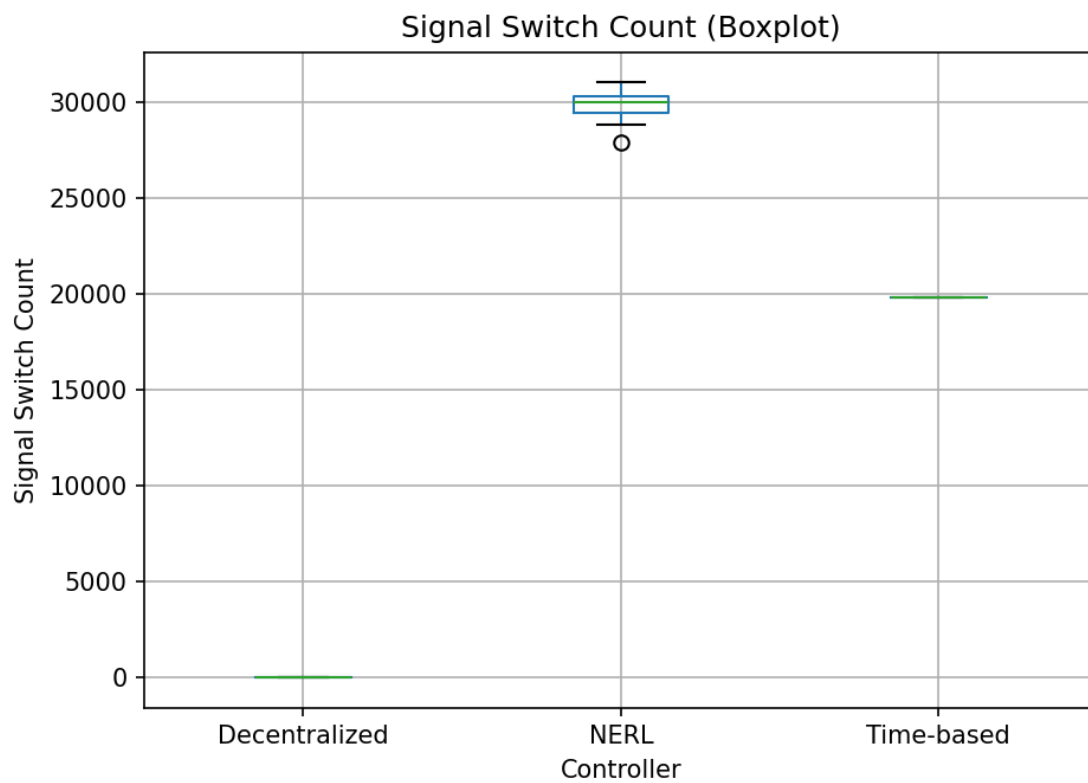


Figure 4.3.4 Comparison of Signal Switch Count Distribution for Each Strategy

As expected, due to its fixed-cycle switching nature, the number of switches for the time-based controller was stable at 19,800. However, the average number of switches for the NERL controller was as high as 29,835, which is **50.7%** more than the time-based

strategy, and this difference is highly statistically significant.

This point is crucial as it clearly demonstrates the **adaptive** and **dynamic** nature of NERL. NERL does not follow fixed rules but makes frequent, subtle adjustments based on real-time traffic conditions. It is this highly active decision-making behavior that enables it to achieve significant energy efficiency optimization by proactively managing traffic and creating smoother flows while maintaining high robot utilization. It can be said that the extra approximately 10,000 signal switches are the “intelligence cost” that the NERL strategy pays to save **13.1%** in energy. However, it should also be noted that overly frequent switching may lead to long-term costs such as hardware wear or decision delays. How to strike a balance between decision benefits and costs is a direction for future research.

4.3.5 Comprehensive Discussion: Analysis of the “Throughput-Energy” Trade-off Space

Synthesizing the analysis of throughput, energy efficiency, and operational dynamics, we can position the three control philosophies in a conceptual “throughput-energy” trade-off space to summarize their core characteristics:

1. **Priority-Based Decentralized Strategy:** Occupies the “high-throughput, high-energy” quadrant. It maximizes the system’s physical potential without regard to energy costs, suitable for scenarios that pursue extreme capacity and are insensitive to operational costs.
2. **Time-Based Strategy:** Positioned in the “medium-throughput, medium-energy” region. Compared to the decentralized strategy, it shows some improvement in energy consumption, but this is accompanied by a loss in throughput, resulting in

a relatively mediocre overall performance that fails to achieve the Pareto Optimal front [30].

3. **NERL Strategy:** Opens up a new “medium-throughput, low-energy” quadrant. Although its throughput is lower than the decentralized strategy, it achieves the lowest energy consumption among the three, and its output is not statistically different from the time-based strategy. Its core contribution is demonstrating that intelligent decision-making can significantly optimize energy efficiency without a substantial sacrifice in capacity, offering a new option for operational models that pursue long-term sustainability and economic benefits, distinct from the traditional “throughput maximization” mindset.

4.4 Chapter Summary

This chapter followed the three-phase experimental procedure established in Chapter 3 to systematically evaluate and compare traffic control strategies representing three different control philosophies.

First, in Section 4.1, by analyzing the cost-effectiveness at different robot densities, **25 robots** was identified as the optimal standard scenario that balances traffic pressure and operational efficiency. Then, in Section 4.2, through parameter optimization of the time-based controller, a **60:40** time allocation ratio was determined as the optimal configuration.

Finally, in the core comparative analysis of Section 4.3, the experimental results revealed the performance characteristics of the different control strategies. In the standard scenario of 25 robots, the NERL controller achieved a significant improvement in system energy efficiency while maintaining an order throughput comparable to the **time-based**

baseline strategy (though significantly lower than the **decentralized** baseline). Its energy per order was statistically significantly lower than both baseline strategies ($p < 0.01$). This result indicates that the NERL method proposed in this study has the potential to find a better benefit balance point in the core “throughput-energy” trade-off than traditional methods.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Research Summary

This study aimed to investigate the trade-off relationship between “system throughput” and “energy efficiency” for different traffic control philosophies in addressing traffic congestion in Robotic Mobile Fulfillment Systems (RMFS). To this end, this paper first established a standardized, medium-density warehouse simulation scenario. On this basis, a systematic comparative analysis was conducted on three representative strategies: adaptive learning (NERL), static centralized (Time-Based), and priority-based decentralized.

According to the experimental results in Chapter 4, this study revealed a clear performance trade-off among the different strategies:

The **priority-based decentralized strategy** established the upper limit for system throughput, with an order completion rate significantly higher than the other two controlled strategies. However, this high output came at the cost of substantial energy consumption, making it representative of a “high-throughput, high-energy” approach.

- The **NERL strategy** demonstrated a commanding advantage in energy efficiency, with its energy per order being statistically significantly lower than both baseline strategies. However, this significant energy saving was achieved by sacrificing some system throughput, as its completion rate was significantly lower than the decentralized strategy.
- The **optimized Time-Based strategy**'s performance on various metrics fell between the other two, offering a relatively balanced but non-optimal solution.

Overall, the core finding of this study is that the NERL controller did not surpass traditional methods on all metrics. Instead, it successfully identified a new, excellent operating point in the “throughput-energy” trade-off space, oriented towards energy efficiency. It proved that intelligent decision-making can achieve significant operational cost (energy) savings with a manageable trade-off in capacity.

5.2 Research Contributions

The main contributions of this study are providing a systematic comparative framework for the RMFS traffic control problem that focuses on the “throughput-energy” trade-off analysis, and validating the unique potential of Neuroevolution Reinforcement Learning (NERL) in finding new benefit balance points for this problem.

1. **Methodological Contribution: Established a standardized multi-objective comparison framework.** The process of this research, from defining a baseline scenario and optimizing a baseline controller to conducting a horizontal comparison on a unified platform, provides a relatively rigorous and reproducible methodology for evaluating different warehouse control strategies in the future.

The core value of this framework is that it shifts the focus of evaluation from a single performance metric (like purely pursuing throughput) to a comprehensive trade-off analysis of multiple, even conflicting, operational objectives (like throughput and energy consumption), which is more aligned with real-world operational needs.

2. **Empirical Contribution: Revealed the unique advantages of NERL in trade-off learning.** The experimental results of this study show that the value of the NERL controller lies not in its complete superiority over traditional strategies, but in its ability to manage complex trade-offs. The study confirmed that, compared to the decentralized strategy that pursues maximum throughput, NERL can significantly reduce energy per order by **19.4%**; and compared to the traditional time-based strategy, it can save **13.1%** of energy while achieving a comparable throughput. This finding provides a strong empirical case for the application of NERL technology in the field of logistics automation. It suggests that adaptive learning algorithms can autonomously learn more refined resource coordination and trade-off strategies that are superior to traditional fixed rules in complex dynamic systems, offering a promising path for developing more efficient and energy-saving intelligent warehouse systems.

5.3 Research Limitations

To ensure the focus and depth of this study, the applicability of its conclusions is also limited by its research boundaries. Acknowledging these limitations will help in objectively understanding the contributions of this study and will point the way for future research.

- **Limitation in Algorithm Breadth:** The core of this study was to compare three different control “philosophies,” not to conduct a large-scale performance competition of algorithms. Therefore, to deeply analyze the potential of NERL in trade-off learning, the research scope did not include a horizontal performance comparison of various deep reinforcement learning algorithms (such as DQN, A3C, etc.).
- **Limitation in Scenario Generalizability:** All conclusions of this study are based on a specific, medium-density warehouse scenario. Although the scenario was designed to be representative, whether the findings of this study, especially regarding the performance ranking and energy-saving potential of different strategies, can be directly generalized to higher or lower density, or to warehouse environments with different physical layouts, remains an open question that requires further empirical research.
- **Independence of Path Planning:** This study assumed that the global paths of all robots were pre-determined by an independent A* algorithm. The core focus was on local conflict coordination at the intersection level and did not involve the possibility of jointly optimizing traffic control with global path planning.
- **Fixity of Hyperparameters:** To ensure consistency in comparison, this study used a fixed set of hyperparameter configurations for the NERL controller. The study did not conduct systematic hyperparameter tuning or ablation studies, so it cannot be claimed that the current configuration is the global optimum.

5.4 Future Work

Based on the findings and existing limitations of this study, future research can be deepened and expanded in the following directions:

1. **Expanding Training Scale to Explore Performance Limits:** Although the performance trend line of the NERL training in this study showed an upward trend after 20 generations, it had not yet reached a fully converged stable state. Therefore, a direct and valuable future research direction is to **significantly extend the number of training generations** (e.g., to 50 or 100 generations) with more powerful computational resources. This would help to more fully explore the upper potential of the NERL model and could verify whether longer training can lead the agent to find a more advantageous Pareto Optimal front [30] in the “throughput-energy” trade-off space.
2. **Cross-Scenario Generalization Ability Verification:** To more comprehensively evaluate the robustness of the NERL controller, future research could deploy the optimal model trained in this study directly into warehouse environments with different robot densities (higher or lower) and different physical layouts for “Zero-shot” or “Fine-tuning” tests. This would allow for a deeper evaluation of its policy’s generalization ability and reveal potential weaknesses of the current model in adapting to new environments.
3. **Joint Optimization of Traffic Control and Global Path Planning:** This study treated path planning and traffic control as two separate modules. However, in the real world, the two are highly coupled. Future research could explore a more ambitious framework, for example, by feeding the congestion level of intersections as a dynamic cost back to the upstream global path planner, thereby constructing a multi-agent learning system that can achieve “path planning-traffic control” joint optimization.
4. **Systematic Hyperparameter and Reward Function Ablation Studies:** This study used a fixed set of hyperparameters and a comprehensive global reward

function. Future research could conduct more detailed ablation studies, for example, by systematically analyzing the impact of key hyperparameters like population size and mutation rate on learning efficiency and final performance; or by conducting step-by-step removal tests of the various penalty terms in the global reward function (such as waiting penalty, stop-and-go penalty, etc.) to quantify the specific contribution of each component in guiding the agent to learn specific trade-off behaviors.

REFERENCES

1. eMarketer. (2024, February). *Worldwide ecommerce will account for over 20% of retail sales in 2024*. <https://www.emarketer.com/content/ecommerce-account-more-than-20-of-worldwide-retail-sales-despite-slowdown>
2. DHL. (2024). *Sustainability trends in logistics for 2024*. <https://www.dhl.com/discover/en-global/logistics-advice/sustainability-and-green-logistics/sustainability-trends-in-logistics>
3. PwC. (2023). *The EU CBAM: Implications for supply chains*. <https://www.pwc.com/gx/en/services/tax/esg-tax/cbam-supply-chain-imperatives.html>
4. Cheng, B., Xie, T., Wang, L., Tan, Q., & Cao, X. (2024). Deep reinforcement learning driven cost minimization for batch order scheduling in robotic mobile fulfillment systems. *Expert Systems with Applications*, 255, 124589.
5. Rizqi, Z. U., Chou, S. Y., & Oscar, A. D. (2025). Towards energy-efficient Robotic Mobile Fulfillment System: Hybrid agent-based simulation with DEA-based surrogate machine learning. *Applied Soft Computing*, 176, 113141.
6. Wurman, P. R., D'Andrea, R., & Mountz, M. (2008). Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI magazine*, 29(1), 9-9.
7. Guizzo, E. (2012, March 19). *Amazon acquires Kiva Systems for \$775 million*. IEEE Spectrum. <https://spectrum.ieee.org/amazon-acquires-kiva-systems-for-775-million>
8. Merschformann, M., Lamballais, T., De Koster, M. B. M., & Suhl, L. (2019). Decision rules for robotic mobile fulfillment systems. *Operations Research Perspectives*, 6, 100128.
9. Stern, R. (2019). Multi-agent path finding—an overview. *Artificial intelligence: 5th RAAI summer school, dolgoprudny, Russia, July 4–7, 2019, tutorial lectures*, 96-115.
10. Li, X., Hua, G., Huang, A., Sheu, J. B., Cheng, T. C. E., & Huang, F. (2020). Storage assignment policy with awareness of energy consumption in the Kiva mobile fulfilment system. *Transportation Research Part E: Logistics and Transportation Review*, 144, 102158.
11. Luo, L., Zhao, N., Zhu, Y., & Sun, Y. (2023). A* guiding DQN algorithm for automated guided vehicle pathfinding problem of robotic mobile fulfillment systems. *Computers & Industrial Engineering*, 178, 109112.
12. Zhou, Y., Guo, K., Yu, C., & Zhang, Z. (2024). Optimization of multi-echelon

- spare parts inventory systems using multi-agent deep reinforcement learning. *Applied Mathematical Modelling*, 125, 827-844.
13. Dunne, M. C., & Potts, R. B. (1964). Algorithm for traffic control. *Operations Research*, 12(6), 870-881.
 14. Teja, V. A., Viswanath, D. K., & Krishna, K. M. (2009, February). A mixed autonomy coordination methodology for multi-robotic traffic control. In *2008 IEEE International Conference on Robotics and Biomimetics* (pp. 1273-1278). IEEE.
 15. Ross, D. W., Sandys, R. C., & Schlaefli, J. L. (1971). A computer control scheme for critical-intersection control in an urban network. *Transportation Science*, 5(2), 141-160.
 16. Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction* (Vol. 1, No. 1, pp. 9-11). Cambridge: MIT press.
 17. Zheng, G., Xiong, Y., Zang, X., Feng, J., Wei, H., Zhang, H., ... & Li, Z. (2019, November). Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1963-1972).
 18. Cao, K., Wang, L., Zhang, S., Duan, L., Jiang, G., Sfarra, S., ... & Jung, H. (2024). Optimization control of adaptive traffic signal with deep reinforcement learning. *Electronics*, 13(1), 198.
 19. Zhu, Y., Cai, M., Schwarz, C. W., Li, J., & Xiao, S. (2022). Intelligent traffic light via policy-based deep reinforcement learning. *International Journal of Intelligent Transportation Systems Research*, 20(3), 734-744.
 20. Sujana, J. A. J., & Namasivaayam, L. (2024, April). Pressure, Queue, and Average Speed-Based Multi-Agent DQN for Optimizing Traffic Signal Control. In *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)* (pp. 1-6). IEEE.
 21. Chen, Y. F., Liu, M., Everett, M., & How, J. P. (2017, May). Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)* (pp. 285-292). IEEE.
 22. Ma, Z., Luo, Y., & Ma, H. (2021, May). Distributed heuristic multi-agent path finding with communication. In *2021 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 8699-8705). IEEE.
 23. Heidrich-Meisner, V., & Igel, C. (2009). Neuroevolution strategies for episodic reinforcement learning. *Journal of Algorithms*, 64(4), 152-168.
 24. Hadhoud, M. (2019). Comparative study of NeuroEvolution algorithms in

- reinforcement learning for self-driving cars. *European Journal of Engineering Science and Technology*.
25. Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*.
 26. Yuan, R., Dou, J., Li, J., Wang, W., & Jiang, Y. (2023). Multi-robot task allocation in e-commerce RMFS based on deep reinforcement learning. *Mathematical biosciences and engineering: MBE*, 20(2), 1903-1918.
 27. Zhou, X., Shi, X., Chu, W., Jiang, J., Zhang, L., & Deng, F. (2024, March). Learning to Solve Multi-AGV Scheduling Problem with Pod Repositioning Optimization in RMFS. In *2024 IEEE International Conference on Industrial Technology (ICIT)* (pp. 1-8). IEEE.
 28. Zou, B., Xu, X., & De Koster, R. (2018). Evaluating battery charging and swapping strategies in a robotic mobile fulfillment system. *European Journal of Operational Research*, 267(2), 733-753.
 29. Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary computation*, 10(2), 99-127.
 30. Deb, K. (2001). Multi-objective optimization using evolutionary algorithms John Wiley & Sons. Inc., New York, NY.
 31. Bao, J., Wu, C., Lin, Y., Zhong, L., Chen, X., & Yin, R. (2023). A scalable approach to optimize traffic signal control with federated reinforcement learning. *Scientific Reports*, 13(1), 19184.
 32. Kolat, M., Kövári, B., Bécsi, T., & Aradi, S. (2023). Multi-agent reinforcement learning for traffic signal control: A cooperative approach. *Sustainability*, 15(4), 3479.
 33. Lee, H., Han, Y., Kim, Y., & Kim, Y. H. (2022). Effects analysis of reward functions on reinforcement learning for traffic signal control. *PLoS one*, 17(11), e0277813.
 34. Bai, H., Cheng, R., & Jin, Y. (2023). Evolutionary reinforcement learning: A survey. *Intelligent Computing*, 2, 0025.
 35. Mei, H., Li, J., Shi, B., & Wei, H. (2023). Reinforcement learning approaches for traffic signal control under missing data. *arXiv preprint arXiv:2304.10722*.